

ETL-SQL Language Reference

A powerful, high-performance ETL (Extract, Transform, Load) engine that blends the simplicity of **Standard SQL** with the flexibility of **Procedural Automation**. Designed for data engineers who need to orchestrate complex data flows without leaving the comfort of a SQL-first environment.

Connection Management

Connectors define how the engine interacts with external data sources.

CREATE CONNECTION

Defines a reusable connection to a source or destination platform. Connections must be defined before using them.

Syntax:

```
-- Traditional (String-based)
CREATE CONNECTION <name> ON <provider>(<'<connection_string>'>) [WITH(<options>)];

-- Structured (Property-based)
CREATE CONNECTION <name> ON <provider>() WITH(<properties...>, <options...>);
```

Connection Types & Options

FLATFILE (or CSV)

For delimited text files.

```
CREATE CONNECTION <name> ON FLATFILE(<'<file_path>'>) [WITH(<options>)];
```

-- OR --

```
CREATE CONNECTION <name> ON FLATFILE() WITH(PATH=<'<file_path>'>, <options>);
```

- **PATH:** The full path to the file. (Required in structured form)
- **HEADER:** **ON**, **OFF** (Default: **ON**).
- **DELIMITER:** **COMMA**, **PIPE**, **TAB**, **SEMICOLON**, **COLON**, **TILDE** or a literal **<char>** (Default: **COMMA**).
- **ROW_DELIMITER:** **LF**, **CR**, **CRLF**, **TILDE**, **SEMICOLON**, **COLON**, **COMMA**, **TAB**, **PIPE** or a literal **<char>** (Default: **CRLF**).
- **ENCODING:** **UTF8**, **ANSI**, **UTF16**, **LATIN1**, **UNICODE** (Default: **UTF8**).
- **TEXT_QUALIFIER:** **DOUBLEQUOTE**, **SINGLEQUOTE** or a literal **<char>**.
- **ESCAPE_CHAR:** Character used to escape delimiters within fields (e.g. **'\"'**).
- **NULL_AS:** **NULL**, **EMPTY**, **BACKSLASH_N** — how null values are represented in the file.
- **DATE_FORMAT:** Custom date parsing format (e.g. **'yyyy-MM-dd'**).
- **START_AT:** Line number to start reading (1-based).
- **END_AT:** Line number to stop reading.
- **COUNT_AT_END:** **ON**, **OFF** — Validates row count at trailer. (Default: **OFF**)
- **STRICT_SCHEMA:** **ON**, **OFF** — enforces column count matching (Default: **OFF**).

- **COMPRESS:** **ON**, **OFF** — transparent GZip support. (Default: **OFF**)
- **ENCRYPT:** **ON**, **OFF** — AES encryption for the file. (Default: **OFF**)
- **PASSWORD:** Password for encryption/decryption. (Required if ENCRYPT=ON)
- **ALGORITHM:** **MD5**, **SHA1**, **SHA2_256**, **SHA2_512** — algorithm to use for encryption/decryption. (Default: **SHA2_256**)
- **KEYFILE:** Path to the private key file for public-key authentication. (Required if ENCRYPT=ON)
- **PASSPHRASE:** The passphrase for the private key file (if any). (Required if ENCRYPT=ON)
- **FORMAT:** **DELIMITED** (Default) or **FIXED**.
- **TEMPLATE:** The name of an in-memory table (e.g. **#temp**) used to define field offsets for **FORMAT='FIXED'**.
- **TRIM:** **ON**, **OFF** — Removes leading/trailing spaces from fields. (Default: **ON** for **FIXED**, **OFF** for **DELIMITED**).

Fixed-Width Data Processing

ETL-SQL supports processing non-delimited, fixed-width text files by leveraging a template table to define the layout.

Setting up a Fixed-Width Connection:

1. **Define the Template:** Create an in-memory table where the column types or metadata specify the widths.
2. **Create the Connection:** Use **FORMAT='FIXED'** and **TEMPLATE=#YourTable**.

Width Definition Rules:

- **Data Type Length:** The engine automatically uses the length from **VARCHAR(N)**, **CHAR(N)**, or **NVARCHAR(N)**.
- **Metadata Tag:** Use the **/* @width: N */** comment to explicitly set a width (overrides data type length).

Example:

```
-- 1. Define the layout
CREATE TABLE #EmployeeLayout (
    ID INT /* @width: 5 */,
    Name VARCHAR(20),
    DeptCode CHAR(3),
    Active BIT /* @width: 1 */
);

-- 2. Connect to the fixed-width file
CREATE CONNECTION employees ON FLATFILE('employees.dat')
WITH (
    FORMAT='FIXED',
    TEMPLATE=#EmployeeLayout,
    HEADER='OFF',
    TRIM='ON'
);
```

```
-- 3. Query as normal
SELECT * FROM employees;
```

[!IMPORTANT] When using `FORMAT='FIXED'`, the `TEMPLATE` option is **mandatory**. The engine will raise an error if it cannot determine the width for every column in the template.

For Microsoft SQL Server. `CREATE CONNECTION <name> ON MSSQL('<connection_string>')`
`[WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON MSSQL() WITH(SERVER='<server>', DATABASE='<database>',
 <options>);`

- **SERVER:** The server name or IP address for the MSSQL connection. (Required in structured form)
- **DATABASE:** The database name for the MSSQL connection. (Required in structured form)
- **TABLE:** Default table context.
- **TRUSTED_CONNECTION:** `TRUE` / `FALSE`. (Default: `FALSE`)
- **USER:** The username for the MSSQL connection. Do not use if `TRUSTED_CONNECTION` is set to `TRUE`.
- **PASSWORD:** The password for the MSSQL connection. Do not use if `TRUSTED_CONNECTION` is set to `TRUE`.
- **USE_SSL:** `TRUE` / `FALSE`. (Default: `FALSE`)
- **TRUST_SERVER_CERTIFICATE:** `TRUE` / `FALSE`. (Default: `FALSE`)
- Supports integrated security and T-SQL pushdown.

POSTGRES (or NPSQL)

For PostgreSQL. `CREATE CONNECTION <name> ON POSTGRES('<connection_string>')`
`[WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON POSTGRES() WITH(HOST='<host>', DATABASE='<database>',
 <options>);`

- **HOST:** The server name or IP address for the POSTGRES connection. (Required in structured form)
- **DATABASE:** The database name for the POSTGRES connection. (Required in structured form)
- **PORT:** Port for the connection. (Default: `5432`)
- **USER:** The username for the connection.
- **PASSWORD:** The password for the connection.
- **TABLE:** Default table context.
- Supports native SQL pushdown.

ORACLE

For Oracle Database. `CREATE CONNECTION <name> ON ORACLE('<connection_string>')`
`[WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON ORACLE() WITH(HOST='<host>', <options>);`

- **HOST:** The server name or IP address for the ORACLE connection. (Required in structured form)

- **PORT:** Port for the connection. (Default: 1521)
- **SERVICE_NAME:** The Oracle service name.
- **TNS_NAME:** The Oracle TNS alias.
- **USER:** The username for the connection.
- **PASSWORD:** The password for the connection.
- **TABLE:** Default table context (e.g. SCHEMA.TABLE).
- Supports PL/SQL pushdown.

MOCKDB

An in-memory mock database designed for script development and testing. It requires no server, no credentials, and no configuration. The engine provides a set of pre-populated tables so you can write and test queries, joins, and ETL logic immediately.

```
CREATE CONNECTION <name> ON MOCKDB();
```

(No connection string or options are required.)

Pre-populated Tables:

Table	Columns
Users	UserID, UserName, Email
Products	ProductID, ProductName, Price
Orders	OrderID, OrderDate, TotalAmount
Employee	ID, Name, Status, Active, first_name, last_name
departments	column1, column2, column3

- All tables are seeded with a small number of sample rows.
- Writes (INSERT, UPDATE, DELETE) are accepted but do not persist between sessions.
- Supports EXECUTE...BEGIN...END blocks with pass-through SQL execution against the mock tables.
- Supports EXPLAIN, schema introspection, and linting (treated as a database-type connector).

[!NOTE] **MOCKDB** is intended for **development and testing only**. It should not be used in production scripts.

Example:

```
-- Develop and test a query without a real database
CREATE CONNECTION mock ON MOCKDB();

SELECT u.UserName, o.TotalAmount
INTO #result
FROM mock.Users AS u
JOIN mock.Orders AS o ON u.UserID = o.OrderID;

SELECT * FROM #result;

-- Test an EXECUTE block
```

```
EXECUTE mock INTO #emp
BEGIN
    SELECT ID, Name FROM Employee WHERE Active = 1;
END
```

PARQUET

For Apache Parquet columnar files. `CREATE CONNECTION <name> ON PARQUET('<file_path>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON PARQUET() WITH(PATH='<file_path>', <options>);`

- **PATH:** The full path to the file. (Required in structured form)
- **COMPRESSION:** `SNAPPY` (Default), `GZIP`, `LZO`, `BROTLI`, `LZ4`, `ZSTD`, `UNCOMPRESSED`.
- **ENCRYPT:** `ON`, `OFF` — encryption for the file. (Default: `OFF`)
- **PASSWORD:** Password for encryption/decryption. (Required if `ENCRYPT=ON`)
- **ALGORITHM:** `MD5`, `SHA1`, `SHA2_256`, `SHA2_512` — algorithm for encryption/decryption. (Default: `SHA2_256`)
- **KEYFILE:** Path to the private/public key for SSH key-pair encryption. (Required if `ENCRYPT=ON`)
- **PASSPHRASE:** Passphrase for the private key file.

AVRO

For Apache Avro files. `CREATE CONNECTION <name> ON AVRO('<file_path>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON AVRO() WITH(PATH='<file_path>', <options>);`

- **PATH:** The full path to the file. (Required in structured form)
- **SCHEMA_FILE:** Path to a `.avsc` schema file.
- **ENCRYPT:** `ON`, `OFF` — encryption for the file. (Default: `OFF`)
- **PASSWORD:** Password for encryption/decryption. (Required if `ENCRYPT=ON`)
- **ALGORITHM:** `MD5`, `SHA1`, `SHA2_256`, `SHA2_512` — algorithm for encryption/decryption. (Default: `SHA2_256`)
- **KEYFILE:** Path to the private/public key for SSH key-pair encryption. (Required if `ENCRYPT=ON`)
- **PASSPHRASE:** Passphrase for the private key file.

EXCEL

For Excel file formats (.xlsx, .xls, .xlsb). `CREATE CONNECTION <name> ON EXCEL('<file_path>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON EXCEL() WITH(PATH='<file_path>', <options>);`

- **PATH:** The full path to the file. (Required in structured form)
- **SHEET:** Specific sheet name (Default: first sheet).
- **HEADER:** `ON`, `OFF` — treat first row as column headers (Default: `ON`).
- **RANGE:** Explicit cell range to read (e.g. `'A1:D100'`).
- **COMPRESS:** `ON`, `OFF` — GZip compress the output file.

- **ENCRYPT:** **ON**, **OFF** — encryption for the file. (Default: **OFF**)
- **PASSWORD:** Password for encryption/decryption. (Required if ENCRYPT=ON)
- **ALGORITHM:** **MD5**, **SHA1**, **SHA2_256**, **SHA2_512** — algorithm for encryption/decryption. (Default: **SHA2_256**)
- **KEYFILE:** Path to the private/public key for SSH key-pair encryption. (Required if ENCRYPT=ON)
- **PASSPHRASE:** Passphrase for the private key file.

JSON

For JSON data files. **CREATE CONNECTION <name> ON JSON('<file_path>') [WITH(<options>)];**
 -- OR --

CREATE CONNECTION <name> ON JSON() WITH(PATH='<file_path>', <options>);

- **PATH:** The full path to the file. (Required in structured form)
- **ROOT_PATH:** JSONPath to the data array (e.g. **\$.Rows**, **\$.data.items**).
- **COMPRESS:** **ON**, **OFF** — transparent GZip support.
- **ENCRYPT:** **ON**, **OFF** — encryption for the file. (Default: **OFF**)
- **PASSWORD:** Password for encryption/decryption. (Required if ENCRYPT=ON)
- **ALGORITHM:** **MD5**, **SHA1**, **SHA2_256**, **SHA2_512** — algorithm for encryption/decryption. (Default: **SHA2_256**)
- **KEYFILE:** Path to the private/public key for SSH key-pair encryption. (Required if ENCRYPT=ON)
- **PASSPHRASE:** Passphrase for the private key file.

XML

For XML data files. **CREATE CONNECTION <name> ON XML('<file_path>') [WITH(<options>)];**
 -- OR --

CREATE CONNECTION <name> ON XML() WITH(PATH='<file_path>', <options>);

- **PATH:** The full path to the file. (Required in structured form)
- **ROOT_PATH:** XPath to the repeating element (e.g. **/Catalog/Book**).
- **COMPRESS:** **ON**, **OFF** — transparent GZip support.
- **ENCRYPT:** **ON**, **OFF** — encryption for the file. (Default: **OFF**)
- **PASSWORD:** Password for encryption/decryption. (Required if ENCRYPT=ON)
- **ALGORITHM:** **MD5**, **SHA1**, **SHA2_256**, **SHA2_512** — algorithm for encryption/decryption. (Default: **SHA2_256**)
- **KEYFILE:** Path to the private/public key for SSH key-pair encryption. (Required if ENCRYPT=ON)
- **PASSPHRASE:** Passphrase for the private key file.

FTP (or FTP_CONN)

For remote file operations over FTP. **CREATE CONNECTION <name> ON FTP('<host_address>') [WITH(<options>)];**
 -- OR --

CREATE CONNECTION <name> ON FTP() WITH(HOST='<host_address>', <options>);

- **HOST:** The FTP server address or IP. (Required in structured form)
- **PORT:** The port for the FTP connection. (Default: **21**)

- **USER:** The username for the FTP connection.
- **PASSWORD:** The password for the FTP connection.
- Supports `GET_FILE`, `PUT_FILE`, and `REMOTE_FILE_LIST`.

SFTP (or SSH)

For remote file operations over SSH. `CREATE CONNECTION <name> ON SFTP('<host_address>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON SFTP() WITH(HOST='<host_address>', <options>);`

- **HOST:** The SSH server address or IP. (Required in structured form)
- **PORT:** The port for the SSH connection. (Default: 22)
- **USER:** The username for the SSH connection.
- **PASSWORD:** The password for the SSH connection. Do not use if KEYFILE is set.
- **KEYFILE:** Path to the private key file for public-key authentication.
- **PASSPHRASE:** The passphrase for the private key file (if any).
- Supports `GET_FILE`, `PUT_FILE`, and `REMOTE_FILE_LIST`.

AZURE_BLOB (or BLOB)

For Azure Blob Storage. `CREATE CONNECTION <name> ON AZURE_BLOB('<host_address_or_ip>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON AZURE_BLOB() WITH(HOST='<host_address_or_ip>', <options>);`

- **CONTAINER:** The target blob container name.
- **ACCOUNT_NAME / ACCOUNT_KEY:** Storage credentials.
- Supports `GET_FILE`, `PUT_FILE`, and `REMOTE_FILE_LIST`.

SMTP (or EMAIL)

For sending emails. `CREATE CONNECTION <name> ON SMTP('<host_address_or_ip>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON SMTP() WITH(HOST='<host_address_or_ip>', <options>);`

- **PORT:** SMTP server port. (Default: 25)
- **USERNAME / PASSWORD:** Authentication details.
- **USE_SSL:** `TRUE` / `FALSE`. (Default: `FALSE`)
- **DEFAULT_FROM:** Default sender email.
- Supports `SEND_EMAIL`.

DIRECTORY

Represents a local file system directory for listing files and performing filesystem operations. `CREATE CONNECTION <name> ON DIRECTORY('<directory_path>') [WITH(<options>)]`;

-- OR --

`CREATE CONNECTION <name> ON DIRECTORY() WITH(PATH='<directory_path>', <options>);`

- **CREATE:** **ON** / **OFF** — Create a new directory if it doesn't exist. (Default: **ON**)
- Supports **COPY_FILE**, **MOVE_FILE**, **DELETE_FILE**, and directory listing via **SELECT**.

Examples:

```
-- Delimited CSV file: Traditional
CREATE CONNECTION csv_in ON FLATFILE('C:\Data\employees.csv')
    WITH(HEADER=ON, DELIMITER=COMMA);

-- Delimited CSV file: Structured
CREATE CONNECTION csv_struct ON FLATFILE()
    WITH(PATH='C:\Data\employees.csv', HEADER=ON, DELIMITER=COMMA);

-- Encrypted and compressed flat file
CREATE CONNECTION secure_file ON FLATFILE('C:\Data\payroll.csv.gz')
    WITH(COMPRESS=ON, ENCRYPT=ON, PASSWORD='s3cr3t');

-- Excel workbook — specific sheet and range
CREATE CONNECTION xl_src ON EXCEL('C:\Reports\Q4.xlsx')
    WITH(SHEET='Summary', HEADER=ON, RANGE='A1:F500');

-- JSON file with nested data path
CREATE CONNECTION json_src ON JSON('C:\Data\orders.json')
    WITH(ROOT_PATH='$.orders');

-- XML file with XPath root
CREATE CONNECTION xml_src ON XML('C:\Data\catalog.xml')
    WITH(ROOT_PATH='/Catalog/Product');

-- Parquet with explicit compression
CREATE CONNECTION parquet_out ON PARQUET('C:\Data\output.parquet')
    WITH(COMPRESS=SNAPPY);

-- SQL Server: Traditional (String-based)
CREATE CONNECTION db ON MSSQL('Server=myserver;Database=DW;Integrated
Security=true')
    WITH(TABLE='dbo.Employees');

-- SQL Server: Structured (Property-based)
CREATE CONNECTION db_new ON MSSQL()
    WITH(SERVER='myserver', DATABASE='DW', TRUSTED_CONNECTION=TRUE,
TABLE='dbo.Employees');

-- PostgreSQL
CREATE CONNECTION pg ON
POSTGRES('Host=localhost;Database=mydb;Username=etl;Password=pass');

-- SFTP: Traditional
CREATE CONNECTION sftp_conn ON SFTP('sftp.example.com')
    WITH(USER='admin', PASSWORD='secret');

-- SFTP: Structured
```



```

CREATE CONNECTION sftp_struct ON SFTP()
    WITH(HOST='sftp.example.com', USER='admin', PASSWORD='secret');

-- SFTP with key-based auth
CREATE CONNECTION sftp_key ON SFTP('sftp.example.com')
    WITH(USER='deploy', KEYFILE='/home/etl/.ssh/id_rsa', PASSPHRASE='keypass');

-- FTP
CREATE CONNECTION ftp_conn ON FTP('ftp.example.com')
    WITH(USER='ftpuser', PASSWORD='ftppass');

-- Azure Blob Storage
CREATE CONNECTION cloud_store ON
    AZURE_BLOB('DefaultEndpointsProtocol=https;AccountName=myaccount;AccountKey=...')
    WITH(CONTAINER='backup');

-- Local directory
CREATE CONNECTION data_dir ON DIRECTORY('C:\Data\Incoming');

-- SMTP email
CREATE CONNECTION mailer ON SMTP('smtp.gmail.com')
    WITH(PORT=587, USERNAME='alerts@example.com', PASSWORD='apppassword',
    USE_SSL=TRUE);

-- Encrypted connection string
CREATE CONNECTION secure_db ON MSSQL('ENC:U2FsdGVkX1+...');

```

DROP CONNECTION

Removes a previously defined connection from the current execution context.

Syntax: **DROP CONNECTION** [**IF EXISTS**] <connection_name>;

Example:

```
DROP CONNECTION IF EXISTS remote_srv;
```

ALTER CONNECTION

Modifies a previously defined connection. Previous options are preserved unless explicitly changed.

Syntax: **ALTER CONNECTION** <connection_name> [**WITH**(<options below separated by commas>)];

Example:

```
ALTER CONNECTION remote_srv WITH(PASSWORD='newpassword');
```

CREATE OR ALTER CONNECTION

Creates a new connection or modifies an existing connection. If the connection exists it will be rebuilt with the new options provided. Previous options are not preserved.

Syntax: `CREATE OR ALTER CONNECTION <connection_name> ON <connection_type> (<connection_string>) [WITH(<options below separated by commas>)];`

Example:

```
CREATE OR ALTER CONNECTION remote_srv ON
MSSQL('Server=myserver;Database=DW;Integrated Security=true')
WITH(TABLE='dbo.Employees');
```

CREATE SSH_KEY_PAIR

Generates a new SSH key pair (public and private) at the specified path. Supports both SQL-style and function-style syntax.

Syntax (SQL Style):

```
CREATE SSH_KEY_PAIR 'path' [WITH(BITS=2048, ALGORITHM='RSA', PASSPHRASE='pwd',
COMMENT='comment')];
```

Syntax (Function Style):

```
SSH_KEY_PAIR('path', [bits], [algorithm], [passphrase], [comment]);
```

Options:

- **BITS:** Key size in bits (Default: 2048).
- **ALGORITHM:** Key algorithm (Default: 'RSA').
- **PASSPHRASE:** Optional passphrase to encrypt the private key.
- **COMMENT:** Optional comment embedded in the public key.

Examples:

```
-- Standard 2048-bit RSA key
CREATE SSH_KEY_PAIR 'C:\Keys\id_rsa';

-- 4096-bit RSA key with a passphrase and comment
CREATE SSH_KEY_PAIR 'C:\Keys\id_rsa_prod'
WITH(BITS=4096, PASSPHRASE='StrongPassword123!', COMMENT='Production ETL
Service Account');

-- Shorthand function-style syntax
SSH_KEY_PAIR('C:\Keys\id_rsa_temp', 2048, 'RSA');
```

Session Management

CLEAR SESSION

Explicitly deletes all temporary files, recovery manifests, and encrypted state associated with the current session. This is recommended for security-critical scripts or to free up disk space after large data operations.

Syntax: **CLEAR SESSION;**

Example:

```
-- Perform sensitive operations
...
-- Cleanup before exiting
CLEAR SESSION;
```

USE DOCKER

Spins up a containerized database instance (MsSql, Postgres, or Oracle) for temporary orchestration or testing. Needs to happen before a connection is created that uses it.

Syntax: **USE DOCKER('<image_name>') [AS <alias>];**

Built-in Variables:

- **<alias>.CONNECTION_STRING**: Returns the dynamically generated connection string for the specified container.
- **DOCKER.CONNECTION_STRING**: Returns the connection string for the last started container (backward compatibility).

Management Commands:

- **<alias> CLOSE;** Stops and disposes of the container.
- **<alias> STOP;** Stops the running container (persists state).
- **<alias> START;** Resumes a stopped container.
- **<alias> PAUSE;** Pauses the container execution.
- **DOCKER CLOSE;** Closes all active Docker containers.
- **CLOSE_DOCKER(<alias>)**: Stops and removes a Docker container.

Function Style Syntax:

- **START_DOCKER('<image>', [<alias>])**
- **STOP_DOCKER('<alias>')**
- **PAUSE_DOCKER('<alias>')**
- **RESUME_DOCKER('<alias>')**

- `CLOSE_DOCKER('<alias>')`

Example:

```
-- Multiple containers with aliases
USE DOCKER('mcr.microsoft.com/mssql/server:2022-latest') AS dms;
DECLARE @conn varchar(500) = dms.CONNECTION_STRING;

USE DOCKER('postgres:15-alpine') AS dpost;
DECLARE @pconn varchar(500) = dpost.CONNECTION_STRING;

-- Management
dms STOP;
dms START;
dms CLOSE;
dpost CLOSE;
```

Variables & State Management

You have the ability to query from multiple different source databases and files. When using these different sources you have access to different processes and syntax. Knowing your context will help to avoid issues.

Walk thru

In the query below your context is the ETL-SQL engine.

```
CREATE CONNECTION c ON FLATFILE('C:\Users\chuck\scratch\ETL-
SQL\TestData\test_categories.csv');
DECLARE @date datetime = '01/01/2026';
SELECT @date;
```

But what if I wanted to run that on the database?

```
CREATE CONNECTION m ON MSSQL('localhost');

EXECUTE m
BEGIN
    DECLARE @date datetime = '01/01/2026';
    SELECT @date;
END
```

Now everything inside the `EXECUTE <name> BEGIN...END` block is run from the context of the SQL Server Engine.

What if I want the results

```
CREATE CONNECTION m ON MSSQL('localhost');

EXECUTE m INTO #emp
BEGIN
    SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > 1;
END
```

Now the ETL-SQL engine will run the query in the EXECUTE block and the results returned from that query will be stored in a temporary table called #emp in the ETL-SQL engine. So the results (#emp) now live in the context of ETL-SQL and as such it can transform, join, load them back into other connections.

In this scenario #emp is a temporary table in the ETL-SQL engine. It is not a table in the Sql Server database. It is a table in the ETL-SQL engine's memory.

What if I need to pass parameters down to the database?

```
CREATE CONNECTION m ON MSSQL('localhost');
DECLARE @id INT = 1;
        ,@name VARCHAR(50) = 'John';

EXECUTE m INTO #emp WITH(@id, @name)
BEGIN
    SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > ? AND t.[name] = ?;
END
```

This pushes the value of @id into the stored procedure as the first parameter. The ? is a placeholder for the parameter. You can add as many parameters as you need to. They will be processed in order.

What if I need to do way harder ones?

```
CREATE CONNECTION m ON MSSQL('localhost');
DECLARE @id INT = 1;
        ,@name varchar(50) = 'John';
        ,@stmt varchar(2000)
SET @stmt = 'SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > ' + @id + '
AND t.[name] = '' + @name + ''';

EXECUTE (
    @stmt
) AT m
```

Dynamic SQL is a powerful tool, but it can be dangerous. It is important to use it with caution. Note the above query could be written with parameters as shown in the previous example this was just shown as an example of how to use dynamic sql.

My query is really straight forward. I just want to select from a table and get the results.

```
CREATE CONNECTION m ON MSSQL('localhost');

SELECT t.id, t.[name] INTO #temp FROM m.dbo.Employee AS t WHERE t.id > 1;
```

There is a shorthand you can use for simple select queries. You just need to add the connection to the table name in the FROM/JOIN clause. In the above example we have to assume it knows what database to connect to from either security or the connection string. The ETL-SQL engine will then run this query against the Sql Server connection and connect to the dbo.Employee table. It will then return the results to the ETL-SQL engine and store them in the #temp table.

Cool but the temp tables in the ETL-SQL engine do not have the correct data types.

```
CREATE CONNECTION m ON MSSQL('localhost');
CREATE TABLE #emp(
    id int
    ,name varchar(50)
);

INSERT INTO #emp (id, name)
SELECT t.id, t.[name] FROM m.dbo.Employee AS t WHERE t.id > 1;

-- OR
INSERT INTO #emp (id, name)
EXECUTE m
BEGIN
    SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > 1;
END
```

You can explicitly define the columns you want to insert into the temp table. This is a good practice because it will prevent errors if the source table changes. You just have to make sure you have the same number of columns and the same data types or it will fail.

Flow Control

EXECUTE (Remote Execution) Executes SQL code on a remote connection. Supports capturing results into local tables and passing parameters for secure, parameterized execution.

Connection Block Form:

```
EXECUTE ds [INTO #target] [WITH (@param1, @param2, ...)]
BEGIN
    -- Remote SQL dialect (e.g. T-SQL for MSSQL)
    SELECT id, name FROM remote_table WHERE category_id = ?1 OR alt_id = ?1;
END;
```

String Literal Form:

```
EXECUTE (
  'SELECT id, name FROM remote_table WHERE category_id = ?1'
) AT ds [INTO #target] [WITH (@cat_id)];
```

Key Parameters:

- **INTO #table:** Streams results from the remote execution directly into a local ETL-SQL memory table.
- **WITH (@vars):** Passes local variables to the remote server.
 - **Sequential:** Use `?` placeholders in the remote SQL; variables are applied in the order they are listed.
 - **Indexed:** Use `?1`, `?2`, etc., to refer to specific parameters in the `WITH` list. This allows using the same parameter multiple times or referring to them out of order.
- **Note:** The string literal form sends raw SQL directly to the server, so it requires valid target server syntax and standard single-quote escaping (`' '`).

PARALLEL The **PARALLEL** keyword allows for concurrent execution of multiple statements or blocks. This is particularly useful for independent data streams that do not have inter-dependencies, such as loading multiple dimension tables simultaneously.

Syntax:

```
PARALLEL
BEGIN
  <statement_1>;
  <statement_2>;
  ...
END
```

Key Characteristics:

- **Non-blocking:** Statements within a **PARALLEL** block are fired concurrently.
- **Wait-all:** The execution engine waits for *all* statements in the block to complete before moving to the next statement outside the block.
- **Isolated Scopes:** Each branch typically operates on its own connection state to avoid race conditions, though global variables are accessible.

Example:

```
-- Load three independent tables in parallel
PARALLEL
BEGIN
  SELECT * INTO #Dimensions_Date FROM src_db.DateDim;
  SELECT * INTO #Dimensions_Product FROM src_db.ProductDim;
  SELECT * INTO #Dimensions_Store FROM src_db.StoreDim;
END

-- Sequential execution resumes here after all three above are finished
```

RUN SCRIPT Executes another ETL-SQL script file, optionally passing parameters. *Syntax:* **RUN SCRIPT** '**<script_path>**' [**WITH** (**@param1** = **val1**, ...)];

Example:

```
RUN SCRIPT 'sub_process.etlsql' WITH (@batchId = 1234, @env = 'PROD');
```

Variable Management

DECLARE

Defines a variable. The data type is optional; if omitted, it defaults to **ANY**. Multiple variables can be declared in a single statement.

```
DECLARE @name STRING = 'Chuck';  
DECLARE @id = 123; -- Optional type (defaults to ANY)  
DECLARE @list LIST = [1, 2, 3], @count INT = 0;
```

SET

Assigns a value to an existing variable.

```
SET @name = 'Charles';
```

USE PASSWORD

Sets the master password for the current session. This password is used to encrypt and decrypt sensitive data within the script (e.g., connection strings marked with **ENC:**). It is **not** the password for individual file connectors (SFTP, FLATFILE, etc.).

When typed interactively the characters are masked as *. Use **SET SHOW_PASSWORD ON** to reveal the password in plain text output.

Syntax: **USE PASSWORD** = '**<password>**';

Example:

```
USE PASSWORD = 'myMasterSecret';  
  
-- Now this encrypted connection string can be decrypted by the engine  
CREATE CONNECTION db ON MSSQL('ENC:U2FsdGVkX1+...');
```


SET SHOW_PASSWORD

Controls whether the password supplied by `USE PASSWORD` is shown in plain text in the console/messages output. Default is `OFF`.

Syntax:

```
SET SHOW_PASSWORD ON;  
SET SHOW_PASSWORD OFF;
```

```
SET SHOW_PASSWORD ON;  
USE PASSWORD = 'mySecret'; -- password visible in output  
SET SHOW_PASSWORD OFF;    -- restore masked mode
```

SET PROFILING

Enables or disables detailed execution profiling. When `ON`, the engine captures millisecond-level metrics for every statement, which can be viewed in the Performance tab of the Terminal Editor (`ui edit`) or the VS Code extension.

Syntax:

```
SET PROFILING ON;  
SET PROFILING OFF;
```

SET WHAT_IF

Enables or disables "dry-run" mode. When `ON`, the engine will suppress all side-effect-producing operations (database writes, file system changes, emails, etc.) while logging the intended actions in yellow text to the messages console. This is essential for validating complex scripts before execution.

Syntax:

```
SET WHAT_IF ON;  
SET WHAT_IF OFF;
```

Behavior:

- **Suppressed:** `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `TRUNCATE`, `BULK INSERT`, `FILE` operations, `DIRECTORY` operations, `SEND_EMAIL`, `DOCKER` actions, and DDL (`CREATE/DROP TABLE/INDEX/PROCEDURE/FUNCTION/CONNECTION`).
- **Allowed:** `SELECT`, `DECLARE`, `SET` (variables), `CREATE CONNECTION`, `PRINT`, `EXECUTE` (local), `IF/WHILE` logic.
- **Logging:** Intended side effects are printed as `WHAT IF: Would [action]...` in yellow.

```
SET WHAT_IF ON;  
-- This will log that it would delete, but won't actually touch the file  
DELETE_FILE 'C:\Data\OldBackup.zip';  
SET WHAT_IF OFF;
```

Supported Data Types

ETL-SQL supports a wide range of data types tailored for ETL operations, variable declarations, and table schemas. Below is a detailed breakdown of the supported types, their default formats, and cast behavior.

1. Numeric Types

- **Types:** `INT`, `INTEGER`, `BIGINT`, `SMALLINT`, `TINYINT`, `DECIMAL`, `NUMERIC`, `MONEY`, `FLOAT`, `REAL`, `DOUBLE`
- **Accepted Literals:** `123`, `-45`, `3.14`
- **Default Behavior:** Inferred from literals automatically. Computations on integers yield integers unless a decimal/float is involved.
- **Cast Behavior:** `CAST('12.5' AS INT)` rounds or truncates depending on context. `CAST('3.14' AS DECIMAL)` converts accurately to a high-precision decimal.

2. Temporal (Date & Time) Types

- **Types:** `DATE`, `DATETIME`, `DATETIME2`, `TIMESTAMP`, `TIME`, `DATETIMEOFFSET`
- **Accepted Literals:** `'2023-10-31'`, `'2023-10-31 15:30:00'`, `'15:30:00'`
- **Default Behavior:** Parsed automatically if the string matches common ISO 8601 formatting, or by using explicit functions like `DATETIMEFROMPARTS`.
- **Cast Behavior:** `CAST('2024-01-01' AS DATE)` parses the string into a `DateTime` struct. Casting a date back to a string uses the standard `yyyy-MM-dd HH:mm:ss` format unless otherwise specified via `FORMAT()`.
- **System Constants:** `SYSDATE`, `CURRENT_TIMESTAMP`, `GETDATE()`, `NOW()` return the current system date/time.
 - **Identifiers:** `SYSDATE` and `CURRENT_TIMESTAMP` are bare identifiers (no parentheses).
 - **Functions:** `GETDATE()` and `NOW()` MUST include parentheses.
- **Date Arithmetic:** (FW-6) Supports shorthand arithmetic for days. `SYSDATE + 1` returns tomorrow. `GETDATE() - 7` returns a date from one week ago. `date1 - date2` returns the difference in days as a decimal.
- **Time Zone Conversion:** Use the `AT TIME ZONE` expression to convert a date-time value between time zones.
 - **Syntax:** `<expression> AT TIME ZONE '<timezone_id>'`
 - **Common Windows Time Zone IDs:**
 - `UTC` (Coordinated Universal Time)
 - `Eastern Standard Time` (New York, Toronto)
 - `Central Standard Time` (Chicago, Mexico City)
 - `Mountain Standard Time` (Denver, Phoenix)
 - `Pacific Standard Time` (Los Angeles, Vancouver)
 - `GMT Standard Time` (London, Dublin)
 - `W. Europe Standard Time` (Berlin, Paris, Rome)
 - `Tokyo Standard Time` (Tokyo, Osaka)
 - **Example:**

```
-- Convert UTC current time to Eastern Standard Time
DECLARE @nyTime = GETDATE() AT TIME ZONE 'Eastern Standard Time';
PRINT @nyTime;
```

```
-- Use a variable for the time zone
DECLARE @tz = 'Pacific Standard Time';
SELECT OrderDate AT TIME ZONE @tz FROM Orders;
```

3. Character & String Types

- **Types:** `STRING`, `VARCHAR`, `NVARCHAR`, `TEXT`, `CHAR`
- **Accepted Literals:** `'Hello World'`, `"ColumnName"` (for identifiers, though some engines permit double quotes for strings depending on dialect config)
- **Default Behavior:** Uses Unicode by default (equivalent to `NVARCHAR` in MSSQL).
- **Cast Behavior:** Any value can be cast to `STRING`. Nulls cast to `NULL`.

4. Logical & Boolean Types

- **Types:** `BIT`, `BOOLEAN`, `BOOL`
- **Accepted Literals:** `TRUE`, `FALSE`, `1`, `0`
- **Default Behavior:** Used naturally in `IF` statements and conditionals.
- **Cast Behavior:** `CAST(1 AS BOOL) -> TRUE`. `CAST('true' AS BOOL) -> TRUE`. `CAST('yes' AS BOOL)` might throw a casting error depending on strictness.

5. Binary

- **Types:** `VARBINARY`, `BINARY`, `IMAGE`, `BLOB`
- **Accepted Literals:** Typically generated via functions like `HASHBYTES` or reading from binary `FILE` connectors.
- **Cast Behavior:** Casting to string typically yields a base64 or hex representation.

6. Structured Types

- **Types:** `JSON`, `XML`
- **Default Behavior:** Can be parsed and manipulated using path extraction functions or loaded directly into relational tables via `SELECT * FROM JSON(...)`.

7. Collections & Specialized Types

- **LIST:** An ordered collection of values. Used with `IN` operators or traversed in `FOREACH` loops.

```
DECLARE @myList LIST = (1, 2, 3);
```

- **PATH:** Used specifically for file system paths or connection URIs. It ensures consistent handling of separators across different operating systems.

```
DECLARE @sourcePath PATH = 'C:\Data\Source\';
```

- **ENCRYPTED**: A semantic type for sensitive data such as connection strings, passwords, or API keys.

```
DECLARE @apiKey ENCRYPTED = 'ENC:U2FsdGVkX1+...';
```

- **UNIQUEIDENTIFIER / UUID**: Represents a globally unique identifier (e.g. **NEWID()**).
- **ANY**: The default type when no type is specified in a **DECLARE** statement. It allows for dynamic type inference based on the assigned value.

```
DECLARE @id = 123; -- Implicitly ANY, inferred as INT
```

Data Type Conversion

ETL-SQL provides two primary functions for converting values between data types.

CAST(expression AS type)

Converts an expression to a target data type.

- If the conversion is impossible (e.g., casting 'ABC' to INT), the engine will throw an **ExecutionException** if the cast logic is strict, or return a best-effort **NULL**.
- Supports all data types listed above including aliased types like **NVARCHAR** and **GUID**.

TRY_CAST(expression AS type)

A safe version of **CAST** that returns **NULL** if the conversion fails, rather than throwing an exception. This is ideal for cleaning messy source data where some rows may contain invalid formats.

Comprehensive Cast Examples:

```
-- NUMERIC TYPES
SELECT CAST('42' AS INT)           AS i;      -- Integer
SELECT CAST('1000' AS BIGINT)      AS bi;     -- 64-bit Long
SELECT CAST(123.456 AS DECIMAL(5,2)) AS d;     -- 123.46 (rounds)
SELECT CAST('100.00' AS MONEY)     AS m;      -- Currency formatting
SELECT CAST('3.14159' AS FLOAT)    AS f;      -- 64-bit Double
SELECT CAST('2.718' AS REAL)       AS r;      -- 32-bit Single
SELECT CAST(255 AS TINYINT)        AS t;      -- 8-bit Byte

-- TEMPORAL TYPES
SELECT CAST('2026-04-08' AS DATE)   AS dt;     -- 2026-04-08
SELECT CAST('15:30:00' AS TIME)     AS tm;     -- 15:30:00
SELECT CAST('2026-04-08 15:30:00 -05:00' AS DATETIMEOFFSET) AS dto;
SELECT CAST(GETDATE() AS NVARCHAR) AS str;    -- '2026-04-28 08:30:00'

-- BINARY & HEX (Converts from Base64 if string)
SELECT CAST('SGVsbG8=' AS VARBINARY) AS b;    -- 0x48656C6C6F ('Hello')
SELECT CAST(0x48656C6C6F AS STRING)  AS s;    -- 'Hello'
```

```
-- LOGICAL
SELECT CAST(1 AS BIT) AS b1; -- TRUE
SELECT CAST('FALSE' AS BOOLEAN) AS b2; -- FALSE

-- SPECIALIZED
SELECT CAST('C:\Temp\' AS PATH) AS p; -- Normalized path object
SELECT CAST('550e8400-e29b-41d4-a716-446655440000' AS GUID) AS uid;
SELECT CAST('{"id": 1}' AS JSON) AS j; -- Parsable JSON string
SELECT CAST('<root/>' AS XML) AS x; -- Parsable XML string
SELECT CAST('[0.1, 0.2, 0.3]' AS VECTOR) AS v; -- Numeric vector array
```

Querying & Filtering

SELECT Fetches, transforms, and projects data from an established connection or temporary memory table.

Supported clauses (Syntactic Order):

- **DISTINCT**: When used after **SELECT**, filters out duplicate rows from the final result set.
- **TOP <expression> [PERCENT] [WITH TIES]**: Caps the number of returned rows (placed after **SELECT**).
 - **PERCENT**: Interprets the count as a percentage of the total result set (rounded up).
 - **WITH TIES**: Includes additional rows that have the same values in the **ORDER BY** columns as the last row in the limited set. Requires an **ORDER BY** clause.
 - Supports literal numbers and variables.
- **INTO <target>**: (ETL specific) Streams the results directly into a destination connection or memory table (tables prefixed with #).
- **FROM <target>**: The source table or connection name. Supports aliases (e.g., **FROM my_conn AS T1**).
- **[INNER | LEFT | RIGHT | FULL | LEFT SEMI | LEFT ANTI] [HASH | LOOP | MERGE] JOIN <target> ON <condition>**: Combines data from multiple sources. You can optionally force a specific execution algorithm (**HASH**, **LOOP**, or **MERGE**) to explicitly optimize streaming performance against massive datasets.
 - **LEFT SEMI JOIN**: Returns rows from the left table where a match exists in the right table.
 - **LEFT ANTI JOIN**: Returns rows from the left table where *no* match exists in the right table.
- **[CROSS | OUTER] APPLY (<subquery>) <alias>**: Correlated subquery join. Allows the subquery to refer to columns from the left side of the apply.
- **WHERE <condition>**: Filters rows based on standard logical evaluations.
- **GROUP BY <columns>**: Aggregates datasets based on unique column pairings.
- **GROUP BY ROLLUP(<columns>)**: Produces a result set with subtotals and a grand total. For **ROLLUP(a, b, c)**, generates groupings (a,b,c), (a,b), (a), and () (grand total). Columns not in the current grouping are **NULL**.
- **GROUP BY CUBE(<columns>)**: Produces a result set for all possible combinations of the specified columns (2ⁿ groupings). Includes individual column subtotals and a grand total.
- **GROUP BY GROUPING SETS(<set1>, <set2>, ...)**: Explicitly specifies which groupings to compute. Each set is a parenthesized, comma-separated column list. An empty set () adds a grand-total row.
- **HAVING <condition>**: Filters result sets *after* **GROUP BY** aggregation has been applied.

- **PIVOT** (<aggregate_func>(<col>) FOR <pivot_col> IN (<values...>)) AS <alias>: Rotates a table-valued expression by turning unique values from one column in the expression into multiple columns in the output.
- **UNPIVOT** (<value_col> FOR <name_col> IN (<cols...>)) AS <alias>: Rotates a table-valued expression from a column-based form into a row-based form.
- **ORDER BY** <column> [ASC|DESC] [, ...]: Sorts the result set. Multiple columns are supported. **ASC** (default) or **DESC**.
 - **Note:** You can also use 1-based column indices (e.g., **ORDER BY 2 ASC** sorts by the second column).
 - Can be used with **OFFSET/FETCH NEXT** for pagination.
- **OFFSET** <expression> [ROWS]: Skips a specific number of rows before returning results. Usually used with **ORDER BY**.
- **FETCH NEXT** <expression> ROWS ONLY: An alternative syntax for **LIMIT**, often used with **OFFSET**.
- **LIMIT** <expression>: Caps the number of returned rows (placed at the end of the query). Supports literal numbers and variables.
- **FOR JSON AUTO | PATH | RAW** [, ROOT('name')] [, INCLUDE_NULL_VALUES] [, WITHOUT_ARRAY_WRAPPER]: Formats results as a JSON string.
 - **AUTO:** Automatically determines the JSON hierarchy.
 - **PATH:** Provides full control over the JSON structure via column aliases (e.g., **col AS "Node.Child"**).
 - **RAW:** Returns each row as a single JSON object.
- **FOR XML AUTO | PATH | RAW** [, ROOT('name')] [, ELEMENTS]: Formats results as an XML string.
 - **AUTO:** Automatically determines the XML hierarchy.
 - **PATH:** Uses column aliases for nesting (e.g., **col AS "Node/Child"**).
 - **RAW:** Returns each row as a <row> element.
 - **ELEMENTS:** Formats column values as nested sub-elements instead of attributes (default).

Example (standard query):

```
SELECT DISTINCT
    category,
    SUM(price) AS TotalSales,
    COUNT(DISTINCT transaction_id) AS UniqueItemsSold
INTO #sales_summary
FROM sales_db.transactions
WHERE created_at >= '2026-01-01'
GROUP BY category
HAVING TotalSales > @threshold
ORDER BY TotalSales DESC
LIMIT 10;
```

Example (ROLLUP — subtotals and grand total):

```
-- Produces per-region/product detail, per-region subtotals, and a grand total
SELECT Region, Product, SUM(Amount) AS Total
FROM #sales
GROUP BY ROLLUP(Region, Product)
```

```
ORDER BY Region, Product;  
-- NULL in Region/Product marks the subtotal / grand-total rows
```

Example (CUBE — all combinations):

```
-- Subtotals by Region, by Product, and grand total  
SELECT Region, Product, SUM(Amount) AS Total  
FROM #sales  
GROUP BY CUBE(Region, Product);
```

Example (GROUPING SETS — explicit groupings):

```
-- Only (Region+Product) detail and Region subtotals; no per-product or grand  
total  
SELECT Region, Product, SUM(Amount) AS Total  
FROM #sales  
GROUP BY GROUPING SETS((Region, Product), (Region));
```

Common Table Expressions (CTE)

CTEs provide a way to define temporary result sets that can be referenced within the scope of a single **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement. They improve readability and support recursive logic.

Syntax:

```
WITH [RECURSIVE] cte_name AS (  
    <query_definition>  
) [, ...]  
<main_statement>;
```

Example (Standard CTE):

```
WITH HighSales AS (  
    SELECT category, SUM(price) AS Total  
    FROM sales_db.transactions  
    GROUP BY category  
)  
SELECT * FROM HighSales WHERE Total > 10000;
```

Example (Recursive CTE):

```
-- Generate a sequence of numbers  
WITH RECURSIVE Counter AS (  
    -- Base case: start with 1
```

```
SELECT 1 AS n
UNION ALL
SELECT n + 1 FROM Counter WHERE n < 10
)
SELECT n FROM Counter;
```

PIVOT and UNPIVOT

Example (PIVOT — rotate rows to columns):

```
-- Sales per quarter, pivoted so each quarter becomes a column
SELECT category, [Q1], [Q2], [Q3], [Q4]
FROM (
    SELECT category, quarter, amount FROM #sales
) AS src
PIVOT (
    SUM(amount) FOR quarter IN ([Q1], [Q2], [Q3], [Q4])
) AS pvt;
```

Example (UNPIVOT — rotate columns back to rows):

```
-- Normalize Q1-Q4 columns back into rows
SELECT category, quarter, amount
FROM #quarterly_sales
UNPIVOT (
    amount FOR quarter IN ([Q1], [Q2], [Q3], [Q4])
) AS unpvt;
```

Pagination

Example (pagination with OFFSET / FETCH NEXT):

```
SELECT id, name, amount
FROM #sales
ORDER BY amount DESC
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

JSON and XML

Example (FOR JSON):

```
SELECT id, name, amount
FROM #sales
FOR JSON PATH, ROOT('Sales'), INCLUDE_NULL_VALUES;
```


Statistical Aggregates

Advanced aggregate functions for statistical analysis. These functions ignore **NULL** values. For paired functions (**CORR**, **COVAR**), rows are excluded if either input is **NULL**.

- **VAR(x) / VAR_SAMP(x)**: Returns the sample variance of a set of numbers.
- **VARP(x) / VAR_POP(x)**: Returns the population variance of a set of numbers.
- **STDEV(x) / STDDEV_SAMP(x)**: Returns the sample standard deviation.
- **STDEVP(x) / STDDEV_POP(x)**: Returns the population standard deviation.
- **COVAR_SAMP(x, y)**: Returns the sample covariance of two sets of numbers.
- **COVAR_POP(x, y)**: Returns the population covariance of two sets of numbers.
- **CORR(x, y)**: Returns the Pearson correlation coefficient between two sets of numbers (range -1.0 to 1.0).

Example:

```
SELECT
  AVG(Price) AS AvgPrice,
  STDEV(Price) AS PriceVolatility,
  CORR(Price, Quantity) AS PriceQuantityCorr
FROM Sales;
```

Logical Operators & Advanced Filters

Standard binary and logical operators are natively supported (**=**, **<**, **>**, **<=**, **>=**, **<>**, **AND**, **OR**).

IN and **NOT IN** Checks a field's value against an explicit list or a **LIST** variable.

```
WHERE category IN ('A', 'B')
      OR status NOT IN @validCats
```

LIKE Matches strings against SQL-standard wildcard patterns (**%** for any string, **_** for any single character). You can optionally use **ESCAPE 'char'** to treat wildcard characters as literal characters.

```
WHERE email LIKE '%@gmail.com'
      AND url LIKE 'https://test\_%' ESCAPE '\'
```

EXISTS and **NOT EXISTS** Evaluates whether a subquery returns any rows.

```
WHERE EXISTS (SELECT 1 FROM #temp WHERE id = main.id)
```

Set Operations

Combine results from multiple queries.

- **UNION ALL**: Combines all rows from both queries.
- **UNION**: Combines rows and removes duplicates.
- **EXCEPT**: Returns rows from the first query that are not in the second.
- **INTERSECT**: Returns rows present in both queries.

Data Manipulation Language (DML)

INSERT INTO

Injects evaluated query outputs directly into a target table or connection.

Syntax: **INSERT INTO** <connection_or_table> [(cols...)] [OUTPUT clauses...] **SELECT** <columns...> **FROM** <source> [WHERE <condition>];

Example:

```
INSERT INTO sales_db.archive (category, TotalSales)
OUTPUT INSERTED.category, INSERTED.TotalSales INTO #AuditLog
SELECT category, TotalSales
FROM #sales_summary
WHERE TotalSales < 1000;
```

UPDATE

Updates pre-existing datasets. Supports **OUTPUT** to capture before/after states via **DELETED** and **INSERTED** pseudo-tables.

Syntax: **UPDATE** <connection.table> **SET** <col> = <val> [OUTPUT clauses...] [WHERE <condition>];

Example:

```
UPDATE sales_db.archive
SET status = 'Closed'
OUTPUT DELETED.status AS OldStatus, INSERTED.status AS NewStatus
WHERE created_at < '2020-01-01';
```

DELETE

Removes rows from a table.

Syntax: **DELETE FROM** <connection.table> [OUTPUT clauses...] [WHERE <condition>];

MERGE (UPSERT)

Synchronizes data between a source and target by performing multiple DML actions (INSERT, UPDATE, DELETE) in a single statement. It is the core of incremental ETL processing.

Syntax:

```
MERGE INTO <target> [AS T]
USING <source_table_or_subquery> [AS S]
ON <match_condition>
[WHEN MATCHED [AND <condition>] THEN <action>]
[WHEN NOT MATCHED [BY TARGET] [AND <condition>] THEN <action>]
[WHEN NOT MATCHED BY SOURCE [AND <condition>] THEN <action>];
```

Supported Actions:

- UPDATE SET col = val, ...
- INSERT (cols...) VALUES (vals...)
- DELETE

Example:

```
MERGE INTO target_table AS T
USING (SELECT * FROM staging_table) AS S
ON T.id = S.id
WHEN MATCHED THEN
    UPDATE SET name = S.name, updated_at = GETDATE()
WHEN NOT MATCHED THEN
    INSERT (id, name) VALUES (S.id, S.name)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

BULK INSERT

Dramatically accelerates data loading directly by streaming payload binaries (such as CSVs) onto target databases or tables using strict destination schema adherence. Features advanced column-reordering maps natively.

Syntax: BULK INSERT <connection_or_table> [(target_cols...)] FROM '<file_path>' WITH (<options>);

Supported Options:

- **FORMAT:** CSV, PARQUET, AVRO, EXCEL (Default: CSV).
- **BATCHSIZE:** Number of rows to commit per transaction (e.g., 10000).
- **MAXERRORS:** Number of parsing errors allowed before the job fails (Default: 0).
- **FIELDTERMINATOR:** Column separator (e.g., ',', '|', '\t').
- **ROWTERMINATOR:** Row separator (e.g., '\n', '\r\n').
- **FIRSTROW:** The 1-based index of the first row to be imported (e.g., 2 to skip a header).
- **DATE_FORMAT:** Custom format for date fields.

- **STRICT_SCHEMA:** **ON**, **OFF** (Default: **ON**). If **OFF**, allows for extra columns or missing columns in the data.

Example:

```
BULK INSERT #Target (Name, Location, Age)
FROM 'TestData\test_bulk_mapped.csv'
WITH (FORMAT = 'CSV');
```

Statements

DDL & Resource Management

CREATE TABLE

Defines a structure for storing data, including support for various constraints.

Syntax:

```
CREATE TABLE <table_name> (
    <col_name> <data_type> [IDENTITY] [PRIMARY KEY] [UNIQUE] [NOT NULL | NULL]
    [DEFAULT <expr>] [CHECK (<expr>)] [REFERENCES <ref_table>(<ref_col>)],
    ...
    [CONSTRAINT <name>] PRIMARY KEY (<cols...>),
    [CONSTRAINT <name>] UNIQUE (<cols...>),
    [CONSTRAINT <name>] CHECK (<expr>),
    [CONSTRAINT <name>] FOREIGN KEY (<cols...>) REFERENCES <ref_table>
    (<ref_cols...>)
);
```

Example:

```
CREATE TABLE Orders (
    OrderId INT IDENTITY PRIMARY KEY,
    OrderDate DATETIME DEFAULT GETDATE(),
    TotalAmount DECIMAL(18,2) NOT NULL CHECK (TotalAmount >= 0),
    CustomerId INT REFERENCES Customers(Id),
    Status VARCHAR(20) DEFAULT 'Pending'
);

-- Table-level multi-column constraint
CREATE TABLE OrderItems (
    OrderId INT,
    LineItem INT,
    PRIMARY KEY (OrderId, LineItem)
);
```

ALTER TABLE

Modifies an existing table's structure.

Syntax:

- `ALTER TABLE <name> ADD <col_definition>;`
- `ALTER TABLE <name> DROP COLUMN <col_name>;`
- `ALTER TABLE <name> RENAME COLUMN <old_name> TO <new_name>;`

DROP TABLE

Deletes a table from the engine's memory or the target data source.

```
DROP TABLE [IF EXISTS] table_name;
```

TRUNCATE TABLE

Efficiently removes all rows from a table. Faster than `DELETE` for large datasets.

Syntax: `TRUNCATE TABLE <connection.table>;`

CREATE INDEX

Creates an index on a table to improve query performance. *Syntax:* `CREATE [UNIQUE] INDEX <index_name> ON <table_name> (<column_name> [ASC|DESC] [, ...]);`

DROP INDEX

Removes an existing index. *Syntax:* `DROP INDEX <table_name>.<index_name>;`

Environment Sets

Environment sets let you define named groups of variable assignments that can be applied to the current session in one step. This is useful for switching between environments (e.g. DEV, QA, PROD) without changing the script logic.

CREATE SETS

Defines a named set of variable assignments. The `BEGIN...END` block contains comma-separated `@variable = value` assignments. The optional `SET WITH_PROMPT ON` directive causes `USE SETS` to ask for confirmation before applying the set in interactive mode.

Syntax:

```
CREATE SETS !<name>
BEGIN
    @variable1 = <expression>,
```

```
    @variable2 = <expression>
  [SET WITH_PROMPT ON;]
END
```

Example:

```
-- Define a DEV environment
CREATE SETS !DEV
BEGIN
    @server      = 'dev-db.internal',
    @database    = 'DevWarehouse',
    @schema      = 'dbo'
END

-- Define a PROD environment with a safety prompt
CREATE SETS !PROD
BEGIN
    @server      = 'prod-db.internal',
    @database    = 'ProdWarehouse',
    @schema      = 'dbo';
    SET WITH_PROMPT ON;
END
```

USE SETS

Applies a previously defined named set, assigning all of its variables to the current session. If the set was created with `SET WITH_PROMPT ON`, the engine will prompt for confirmation before applying it (in interactive mode). In batch/non-interactive mode, the set is applied automatically.

Syntax:

```
USE SETS !<name>;
```

Example:

```
DECLARE @server    VARCHAR(100);
DECLARE @database  VARCHAR(100);
DECLARE @schema    VARCHAR(50);

CREATE SETS !DEV
BEGIN
    @server      = 'dev-db.internal',
    @database    = 'DevWarehouse',
    @schema      = 'dbo'
END

-- Define a PROD environment with a safety prompt
```

```

CREATE SETS !PROD
BEGIN
    @server    = 'prod-db.internal',
    @database  = 'ProdWarehouse',
    @schema    = 'dbo';
    SET WITH_PROMPT ON;
END

USE SETS !DEV;

-- @server, @database, and @schema are now set to the DEV values
SELECT @server, @database, @schema;

```

DROP SETS

Removes a named set from the current session.

Syntax:

```
DROP SETS [IF EXISTS] !<name>;
```

Example:

```

DROP SETS !DEV;
DROP SETS IF EXISTS !STAGING; -- No error if it doesn't exist

```

Full environment-switching example:

```

DECLARE @conn VARCHAR(200);
DECLARE @mode VARCHAR(50) = 'unknown';

CREATE SETS !DEV
BEGIN
    @conn = 'Server=dev-db;Database=Dev;',
    @mode = 'development'
END

CREATE SETS !PROD
BEGIN
    @conn = 'Server=prod-db;Database=Prod;',
    @mode = 'production';
    SET WITH_PROMPT ON;
END

-- Switch to DEV (no prompt)
USE SETS !DEV;
CREATE CONNECTION src ON MSSQL(@conn);

```

```
-- Switch to PROD (will prompt in interactive mode)
USE SETS !PROD;
CREATE CONNECTION dest ON MSSQL(@conn);
```

Engine Configuration

SET WHAT_IF

Toggles the engine's "dry-run" mode. When **ON**, all destructive operations (e.g., **INSERT**, **UPDATE**, **DELETE**, **FILE_OPERATION**, **SEND_EMAIL**) are logged to the console but not actually executed. This is useful for validating script logic, variable evaluation, and lineage tracing without modifying data or external resources.

Syntax: **SET WHAT_IF { ON | OFF };**

Default: **OFF**

Example:

```
-- Safely test a delete operation
SET WHAT_IF ON;

DELETE FROM m.dbo.ArchiveEmployees
WHERE TerminatedDate < '2024-01-01';

-- Lineage is still tracked, and logs will show which rows WOULD be deleted.
-- Set back to OFF to perform actual data modifications.
SET WHAT_IF OFF;
```

Control Flow

IF...ELSE

Conditional execution.

```
IF @val > 10
BEGIN
    PRINT 'High';
END
ELSE
BEGIN
    PRINT 'Low';
END
```

WHILE

Loop execution.


```
WHILE @i < 10
BEGIN
    SET @i = @i + 1;
    IF @i = 5 CONTINUE;
    IF @i = 8 BREAK;
END
```

BREAK / CONTINUE

Control the flow of a **WHILE** loop. **BREAK** exits the loop immediately, and **CONTINUE** skips to the next iteration.

TRY...CATCH

Error handling block.

```
BEGIN TRY
    -- potentially failing code
END TRY
BEGIN CATCH
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH
```

RAISERROR / THROW

Manually raise an error.

```
RAISERROR('Custom error message', 16, 1);
-- or
THROW 50000, 'Error message', 1;
```

WAITFOR

Pauses script execution until a specified delay has elapsed or a specific clock time is reached. This is essential for coordinating batch windows or introducing pauses between high-frequency API calls.

Syntax:

- WAITFOR DELAY** '<hh:mm:ss[.fff]>';
- WAITFOR TIME** '<hh:mm:ss[.fff]>';

Key Behaviors:

- DELAY:** Pauses for a fixed duration. Minimum interval is 1ms. Supports milliseconds via decimal fractions (e.g. '00:00:03.500').
- TIME:** Pauses until the system clock reaches the specified time.

- **Intra-day:** If the time is later today, it waits for it.
- **Cross-day:** If the specified time has already passed for the current day, the engine automatically waits until that time **tomorrow**.

Examples:

```
-- Wait for 5 seconds
WAITFOR DELAY '00:00:05';

-- Wait for 100 milliseconds
WAITFOR DELAY '00:00:00.100';

-- Pause until 11:30 PM
WAITFOR TIME '23:30:00';

-- Cross-day example:
-- Assume it is currently 10:00 AM. This will wait 23 hours until 9:00 AM
tomorrow.
WAITFOR TIME '09:00:00';

-- Dynamic delay using a variable
DECLARE @sleep = '00:00:02';
WAITFOR DELAY @sleep;
```

Data Movement & Transformation

EXECUTE / EXEC

Executes a stored procedure or a dynamic SQL block.

```
EXECUTE RemoteProc @param1 = 'val';
EXEC('SELECT * FROM Table') AT RemoteConn;
```

INSERT INTO

Appends data to a table.

```
INSERT INTO TargetTable SELECT * FROM SourceTable;
```

Job & Profile Management

CREATE JOB

Schedules a script to run automatically on a repeating interval.

Syntax:

```
CREATE JOB <job_name> ON SCHEDULE EVERY <n> SECONDS|MINUTES|HOURS|DAYS [AT  
'<HH:MM>'] AS <statement>;
```

- **EVERY <n> SECONDS/MINUTES/HOURS/DAYS**: The interval between runs.
- **AT 'HH:MM'**: Optional. Pin the job to a specific time of day (e.g., run daily at 02:00 midnight).
- **AS <statement>**: The ETL-SQL statement or block to execute. Typically a **RUN SCRIPT** or **BEGIN...END** block.

Examples:

```
-- Run a cleanup script every 30 minutes  
CREATE JOB CleanupJob ON SCHEDULE EVERY 30 MINUTES AS  
    RUN SCRIPT 'scripts/cleanup.etlsql';  
  
-- Run a daily archive job at 2 AM  
CREATE JOB NightlyArchive ON SCHEDULE EVERY 1 DAY AT '02:00' AS  
BEGIN  
    INSERT INTO archive_db.logs SELECT * FROM prod_db.logs WHERE log_date <  
DATEADD(DAY, -30, GETDATE());  
    DELETE FROM prod_db.logs WHERE log_date < DATEADD(DAY, -30, GETDATE());  
END;
```

DROP JOB

Removes a scheduled job.

```
DROP JOB [IF EXISTS] <job_name>;
```

SHOW JOBS

Lists all currently registered background jobs.

```
SHOW JOBS;
```

SHOW JOB HISTORY

Displays the execution history and status of scheduled jobs.

```
SHOW JOB HISTORY;           -- all jobs  
SHOW JOB HISTORY NightlyArchive; -- specific job
```

KILL JOB

Terminates a running background job by its ID.

```
KILL JOB 'job_id_123';
```

SET PROFILING

Enables or disables performance profiling for the session.

```
SET PROFILING ON;  
-- run scripts  
SET PROFILING OFF;
```

SHOW PROFILE

Displays the timing and resource usage for the most recently executed statements.

```
SHOW PROFILE [INTO #temp];
```

SHOW CONNECTIONS

Lists all currently active data source connections in the session.

```
SHOW CONNECTIONS [INTO #temp];
```

SHOW TABLES

Lists all tables available on a specific connection or all connections if omitted.

```
SHOW TABLES [ON <connection>] [INTO #temp];
```

SHOW COLUMNS

Lists columns and (where available) data types for a specific table.

```
SHOW COLUMNS FOR [connection.]table [INTO #temp];
```

SHOW TAGS

Lists metadata tags associated with a table or a specific column (calculated via lineage).

```
SHOW TAGS FOR TABLE <tableName> [COLUMN <columnName>] [INTO #temp];
```

SHOW TAG VALUE

Retrieves the value of a specific metadata tag for a table or column.

```
SHOW TAG VALUE FOR TABLE <tableName> [COLUMN <columnName>] WITH TAG <tagName>
[INTO #temp];
```

Functions

Conditional Logic

CASE ... WHEN ... THEN ... ELSE ... END Evaluates conditions sequentially to return specific projected outputs natively.

```
SELECT
  CASE
    WHEN id < 1000 THEN 'Legacy'
    WHEN id >= 1000 THEN 'Modern'
    ELSE 'Unknown'
  END AS SystemType
FROM system_table;
```

String Functions

- **UPPER(str)**: Returns the string in all-caps.
- **LOWER(str)**: Returns the string in all-lowercase.
- **LEN(string) / LENGTH(string|list)**: Returns the character count of a string or the number of items in a list.
- **TRIM(str)**: Removes leading and trailing whitespaces.
- **LTRIM(str), RTRIM(str)**: Removes leading or trailing whitespaces.
- **REVERSE(str)**: Reverses the characters in the string.
- **CONCAT(s1, s2, ...)**: Concatenates multiple strings into one.
- **SUBSTRING(str, start, length) / SUBSTR(str, start[, length])**: Extracts a portion of a string. Supports ANSI syntax: **SUBSTRING(str FROM start [FOR length])**.
- **TRIM(str)**: Removes leading/trailing spaces. Supports ANSI syntax: **TRIM([BOTH|LEADING|TRAILING] [chars FROM] str)**.
- **LTRIM(str), RTRIM(str)**: Simple leading or trailing space removal.
- **POSITION(sub IN str) / INSTR(str, sub)**: Returns the 1-based index of a substring.

- **OVERLAY(str PLACING overlay FROM start [FOR length])**: Replaces a portion of a string with another string.
- **STRING_SPLIT(str, separator)**: Splits a string into a list of substrings.
- **STRING_ESCAPE(text, type)**: Escapes special characters (e.g. 'json').
- **ASCII(str), UNICODE(str)**: Returns the numeric code of the first character.
- **CHAR(n)**: Converts an ASCII/Unicode code to a character.
- **FORMAT(val, fmt)**: Formats a value based on a .NET format string.
- **PATINDEX(pattern, str)**: Returns the 1-based start position of a pattern in a string.
- **STR(float[, len[, dec]])**: Returns character data converted from numeric data.
- **QUOTENAME(str[, char])**: Returns a delimited identifier (default []).
- **TRANSLATE(str, from, to)**: Replaces characters specified in **from** with **to**.
- **REPLICATE(str, n)**: Repeats a string **n** times.
- **DATALength(val)**: Returns the number of bytes used to represent any expression.
- **TO_STR(val)**: Converts any value to a string.
- **INITCAP(str)**: Capitalizes the first letter of each word.

Logical & System Functions

- **COALESCE(v1, v2, ...)**: Returns the first non-null value in the list.
- **ISNULL(v1, v2) / NVL(v1, v2) / IFNULL(v1, v2)**: Returns **v2** if **v1** is null.
- **NULLIF(v1, v2)**: Returns **NULL** if **v1** equals **v2**, else returns **v1**.
- **IS_NULL(expr)**: Returns **TRUE** if the expression is null.
- **IS_NOT_NULL(expr)**: Returns **TRUE** if the expression is NOT null.
- **IIF(cond, true_val, false_val)**: Returns one of two values depending on a condition.
- **DECODE(val, search1, result1, ..., default)**: Oracle-style CASE shorthand.
- **CAST(expr AS type)**: Converts an expression to a target data type (see [Data Types](#)).
- **TRY_CAST(expr AS type)**: Similar to **CAST**, but returns **NULL** if conversion fails instead of throwing an error.
- **CHECKSUM(v1, v2, ...)**: Returns a 64-bit hash of the input values.
- **BINARY_CHECKSUM(v1, v2, ...)**: Returns a binary-compatible hash.
- **HASHBYTES('algo', val)**: Returns a cryptographic hash (MD5, SHA1, SHA256, SHA512).
- **NEWID() / NEWSEQUENTIALID()**: Returns a new time-ordered unique identifier (UUID v7).

List & Collection Functions

- **LENGTH(list|string)**: Returns the number of items in a list or characters in a string.
- **SORT_LIST(list[, 'ASC' | 'DESC'])**: Returns a sorted version of the list.
- **APPEND_TO_LIST(@list, value) / ADD_TO_LIST**: Adds an item to a list variable.
- **REMOVE_FROM_LIST(@list, value)**: Removes all occurrences of a value from a list variable.
- **GENERATE_SERIES(start, stop[, step])**: Generates a numeric sequence.
- **LEAST(v1, v2, ...), GREATEST(v1, v2, ...)**: Returns the smallest or largest value from a list of arguments.

Math Scalars

- **ROUND(numeric, decimals)**: Rounds a numeric value to a specified number of decimal places.
- **ABS(numeric)**: Returns the absolute value of a number.
- **CEILING(numeric)**: Returns the smallest integer greater than or equal to the number.

- **FLOOR(numeric)**: Returns the largest integer less than or equal to the number.
- **SQRT(float)**: Returns the square root of a number.
- **POWER(base, exp)**: Returns the result of a base raised to an exponent.
- **RAND([seed])**: Returns a pseudo-random float between 0 and 1. If a seed is provided, the sequence is deterministic.
- **SIGN(numeric)**: Returns the sign of a number: **1** for positive, **-1** for negative, and **0** for zero.
- **SIN(float), COS(float), TAN(float)**: Standard trigonometric functions. **Requires input in radians.**
- **ASIN(float), ACOS(float), ATAN(float)**: Inverse trigonometric functions. **Returns results in radians.**
- **ATAN2(y, x)**: Returns the angle in radians between the positive x-axis and the point (x, y).
- **EXP(n)**: Returns **e** raised to the power of **n**.
- **LOG(n), LN(n)**: Returns the natural logarithm (base **e**) of a number.
- **LOG10(n)**: Returns the base-10 logarithm of a number.
- **MOD(n, d)**: Returns the remainder (modulus) of **n** divided by **d**.

Examples:

```
-- Trigonometry (converting degrees to radians)
DECLARE @deg = 90.0;
DECLARE @rad = @deg * (3.14159 / 180.0);
SELECT SIN(@rad) AS Sine90; -- Returns ~1.0

-- Inverse Trig
SELECT ASIN(1.0) AS AngleInRadians; -- Returns ~1.5708 (pi/2)
SELECT ATAN2(10.0, 10.0) AS Angle45; -- Returns ~0.785 (pi/4)

-- Logarithms & Power
SELECT POWER(2, 10) AS Kilobyte; -- 1024
SELECT LOG(2.71828) AS NaturalLog; -- ~1.0

-- Sign & Abs
SELECT SIGN(-15.5) AS s1, ABS(-15.5) AS a1; -- -1, 15.5
```

Statistical Functions

- **SUM(expression)**: Returns the sum of all values in the numeric collection.
- **AVG(expression)**: Returns the average (mean) of all values in the numeric collection.
- **MIN(expression), MAX(expression)**: Returns the minimum or maximum value in the collection.
- **COUNT(expression)**: Returns the number of items in the collection.
- **STDDEV(expression)**: Returns the statistical standard deviation of the population.
- **VAR(expression)**: Returns the statistical variance of the population.

Date and Time Functions

- **GETDATE()** / **NOW()**: Returns the current system date and time.
- **DATEADD(datepart, number, date)**: Adds a specific interval (e.g., **DAY**, **MONTH**, **YEAR**, **hour**) to a date.
- **DATEDIFF(datepart, start, end)**: Returns the count of specified datepart boundaries crossed between two dates.

- **DATEPART(datepart, date)**: Returns an integer representing the specified date part.
- **DATENAME(datepart, date)**: Returns a string representing the specified date part (e.g. 'January').
- **EXTRACT(field FROM source)**: Extracts a component (e.g., YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, DOW, DOY) from a datetime.
- **YEAR(date), MONTH(date), DAY(date)**: Returns the year, month, or day part of a date as an integer.
- **EOMONTH(date[, months_to_add])**: Returns the last day of the month containing the date.
- **ISDATE(expr)**: Returns 1 if the expression is a valid date, 0 otherwise.
- **DATETIMEFROMPARTS(y, m, d, h, mi, s, ms)**: Constructs a DATETIME from its parts.
- **TIMEFROMPARTS(h, mi, s, frac, prec)**: Constructs a TIME from its parts.
- **DATETIMEOFFSETFROMPARTS(y, m, d, h, mi, s, ms, off_h, off_m, prec)**: Constructs a DATETIMEOFFSET from its parts.
- **TRUNC(date) / TO_DATE(date)**: Truncates the time portion from a datetime.
- **expr AT TIME ZONE 'timezone_id'**: Converts a datetime to the specified timezone.

JSON Functions

- **JSON_VALUE(json, path) / JSON_EXTRACT**: Extracts a scalar value from a JSON string at the given path.
- **JSON_QUERY(json, path)**: Extracts an object or array fragment from a JSON string.
- **JSON_MODIFY(json, path, val)**: Updates or inserts a value in a JSON string.
- **ISJSON(str)**: Returns 1 if the string is valid JSON, 0 otherwise.
- **JSON_EXISTS(json, path)**: Returns 1 if the path exists in the JSON string.
- **JSON_OBJECT(k1, v1, k2, v2, ...)**: Constructs a JSON object from key/value pairs.
- **JSON_ARRAY(v1, v2, ...)**: Constructs a JSON array from the provided values.
- **JSON_TABLE(json, path)**: (Table-valued) Expands a JSON array or object into a table.
- **OPENJSON(json[, path])**: (Table-valued) Expands JSON into a table (SQL Server style).
- **JSON_EXTRACT(json, path)**: Returns the data at the specified path (alias for JSON_QUERY).

XML Functions

XML Functions

- **XMLVALUE(xml, xpath) / EXTRACTVALUE**: Extracts a scalar value from an XML string using an XPath expression.
- **XMLEXISTS(xml, xpath)**: Returns 1 if the XPath expression matches any node in the XML.
- **XMLQUERY(xml, xpath)**: Returns an XML fragment from the string using XPath.
- **XMLTABLE(xml, xpath)**: (Table-valued) Expands an XML document into a table based on the row-level XPath.
- **XMLELEMENT(name, contents)**: Constructs an XML element with the given name and child content.
- **XMLATTRIBUTES(n1, v1, n2, v2, ...)**: Constructs XML attributes for an element.
- **XMLFOREST(n1, v1, n2, v2, ...)**: Constructs a forest of XML elements from name/value pairs.

Regular Expressions (Regex)

Regular Expression Functions

- **REGEXP_LIKE(str, pattern[, flags])**: Returns 1 if the string matches the pattern.

- **REGEXP_SUBSTR(str, pattern[, pos[, occ[, flags]]])**: Extracts a substring matching the pattern.
- **REGEXP_REPLACE(str, pattern, new_str[, pos[, occ[, flags]]])**: Replaces matching substrings.
- **REGEXP_INSTR(str, pat[, pos[, occ[, option[, flags]]])**: Returns the 1-based position of a match.
- **REGEXP_COUNT(str, pattern[, pos[, flags]])**: Returns the number of matches found.
- **REGEXP_MATCHES(str, pattern)**: Returns a table of all matches found.
- **REGEXP_SPLIT_TO_TABLE(str, pattern)**: (Table-valued) Splits a string into a table using regex.

Additional Scalar & Math Functions

- **EXP(n), LOG(n), LN(n)**: Exponential and logarithmic functions.
- **MOD(n, m)**: Returns the remainder of n/m .
- **NVL(val, default), NVL2(val, if_not_null, if_null)**: Oracle-style null handling.
- **CUME_DIST(), PERCENT_RANK(), NTH_VALUE(col, n)**: Advanced window and distribution functions.
- **PERCENTILE_CONT(n), PERCENTILE_DISC(n)**: Statistical percentile calculations.

File & Directory Introspection

Functions to query the state of the local or remote filesystem.

- **FILE_EXISTS('path')**: Returns true if the file exists.
- **DIRECTORY_EXISTS('path')**: Returns true if the directory exists.
- **FILE_LIST('path'[, 'filter'])**: Returns a table containing information about files in a directory.
- **REMOTE_FILE_LIST(conn_name [, 'path'])**: Returns a table of files from a remote connection (SFTP, FTP, or Azure Blob). The **conn_name** is the name of a configured remote connector.

```
-- List all files in an SFTP directory
SELECT * FROM REMOTE_FILE_LIST('my_sftp', '/uploads');

-- List files using a configured Azure Blob connection
SELECT FileName, Size FROM REMOTE_FILE_LIST('cloud_store', 'backups/');
```

File Transfer

Transfers files between the local machine and a remote connector (SFTP, FTP, Azure Blob).

SEND FILE

Uploads a local file to a remote connection.

Syntax (SQL Style):

```
SEND FILE '<local_path>' TO '<remote_path>' AT <connection_name>
[WITH(OVERWRITE=ON|OFF)];
```

Syntax (Function Style):

```
SEND_FILE('<local_path>', <connection_name>, '<remote_path>' [,OVERWRITE=ON|OFF]);
```

Example:

```
SEND FILE 'C:\Exports\report.csv' TO '/uploads/report.csv' AT my_sftp  
WITH(OVERWRITE=ON);
```

RECEIVE FILE

Downloads a file from a remote connection to the local machine.

Syntax (SQL Style):

```
RECEIVE FILE FROM '<remote_path>' TO '<local_path>' AT <connection_name>  
[WITH(OVERWRITE=ON|OFF)];
```

Syntax (Function Style):

```
RECEIVE_FILE('<remote_path>', <connection_name>, '<local_path>' [,  
OVERWRITE=ON|OFF]);
```

Example:

```
RECEIVE FILE FROM '/data/input.csv' TO 'C:\Imports\input.csv' AT my_sftp  
WITH(OVERWRITE=OFF);
```

Email Operations

SEND EMAIL

Sends an automated email (requires a configured **SMTP** connection). Supports flexible, any-order SQL-style block syntax.

Syntax (SQL Style):

```
SEND EMAIL  
  TO '<to_address>'  
  FROM '<from_address>'  
  SUBJECT '<subject>'
```

```
BODY '<body>'
[CC '<cc_address>' [, '<cc2>', ...]]
[BCC '<bcc_address>' [, '<bcc2>', ...]]
[ATTACH '<file_path>' [, '<file2>', ...]]
[AT <smtp_connection>];
```

Syntax (Function Style):

```
SEND_EMAIL(<smtp_connection>, '<to>', '<from>', '<subject>', '<body>' [, '<cc>',
'<bcc>', '<attach>']);
```

- **TO** (required): Recipient email address.
- **FROM** (required): Sender email address.
- **SUBJECT** (required): Email subject line.
- **BODY** (required): Email body text.
- **CC**: One or more carbon-copy recipients.
- **BCC**: One or more blind carbon-copy recipients.
- **ATTACH**: One or more local file paths to attach.
- **AT**: The name of the **SMTP** connection to use.

Example:

```
CREATE CONNECTION mailer ON SMTP('smtp.company.com')
  WITH(PORT=587, USERNAME='alerts@company.com', PASSWORD='secret',
  USE_SSL=TRUE);

-- Clauses can be in any order
SEND EMAIL
  FROM 'alerts@company.com'
  TO 'admin@company.com'
  SUBJECT 'ETL Job Completed'
  BODY 'All records processed successfully.'
  ATTACH 'C:\Reports\summary.xlsx'
  AT mailer;
```

Aggregation Functions

Used in **SELECT** statements with an optional **GROUP BY** clause.

- **COUNT**([**DISTINCT**] **col**): Returns the count of non-null items in a collection or group.
- **SUM**(**col**): Returns the total sum of numeric values.
- **AVG**(**col**): Returns the mathematical mean of numeric values.
- **MIN**(**col**), **MAX**(**col**): Returns the smallest or largest value in a set.
- **STDDEV**(**col**), **VAR**(**col**): Returns the statistical standard deviation or variance.
- **STRING_AGG**(**col**, **separator**) [**WITHIN GROUP (ORDER BY col [ASC|DESC])**]: Concatenates values from multiple rows into a single string, separated by the specified string.

Window Functions

Window functions operate on a set of rows and return a single value for each row from the underlying query. The **OVER** clause defines the window or user-specified set of rows. The framing clause is optional and is used to define the window or user-specified set of rows.

Window Framing (**ROWS** | **RANGE**)

Window functions support framing to restrict the rows within the partition used for calculation.

- **ROWS BETWEEN <start> AND <end>**: Specifies a physical offset from the current row.
- **RANGE BETWEEN <start> AND <end>**: Specifies a logical range based on values.

Bounds:

- **UNBOUNDED PRECEDING / FOLLOWING**
- **<n> PRECEDING / FOLLOWING**
- **CURRENT ROW**

```
SELECT
    id,
    amount,
    SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total,
    AVG(amount) OVER(ORDER BY id ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS
moving_avg
FROM #sales;
```

COUNT(col) OVER (ORDER BY col [ASC|DESC])

Counts the number of rows in the window.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    COUNT(OrderID) OVER (ORDER BY OrderDate) AS RunningCount
FROM Orders;
```

SUM(col) OVER (ORDER BY col [ASC|DESC])

Calculates the running total of a column.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
```

```
SUM(TotalAmount) OVER (ORDER BY OrderDate) AS RunningTotal
FROM Orders;
```

MIN(col) OVER (ORDER BY col [ASC|DESC])

Calculates the running minimum of a column.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    MIN(TotalAmount) OVER (ORDER BY OrderDate) AS RunningMin
FROM Orders;
```

MAX(col) OVER (ORDER BY col [ASC|DESC])

Calculates the running maximum of a column.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    MAX(TotalAmount) OVER (ORDER BY OrderDate) AS RunningMax
FROM Orders;
```

AVG(col) OVER (ORDER BY col [ASC|DESC])

Calculates the running average of a column.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    AVG(TotalAmount) OVER (ORDER BY OrderDate) AS RunningAvg
FROM Orders;
```

ROW_NUMBER() OVER (ORDER BY col [ASC|DESC])

Assigns a unique sequential integer to each row within a partition, starting from 1.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
```

```
ROW_NUMBER() OVER (ORDER BY OrderDate) AS RowNum
FROM Orders;
```

RANK() OVER (ORDER BY col [ASC|DESC])

Assigns a rank to each row within a partition based on the specified order. Rows with the same value receive the same rank, and the next rank is skipped (e.g., 1, 1, 3, 4).

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    RANK() OVER (ORDER BY OrderDate) AS Rank
FROM Orders;
```

DENSE_RANK() OVER (ORDER BY col [ASC|DESC])

Assigns a rank to each row within a partition based on the specified order. Rows with the same value receive the same rank, but the next rank is not skipped (e.g., 1, 1, 2, 3).

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    DENSE_RANK() OVER (ORDER BY OrderDate) AS DenseRank
FROM Orders;
```

LAG(col[, offset[, default]]) and LEAD(col[, offset[, default]])

Accesses data from a previous or subsequent row in the same result set without the use of a self-join.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    LAG(TotalAmount, 1, 0) OVER (ORDER BY OrderDate) AS PreviousAmount,
    LEAD(TotalAmount, 1, 0) OVER (ORDER BY OrderDate) AS NextAmount
FROM Orders;
```

FIRST_VALUE(col) and LAST_VALUE(col)

Returns the first or last value in an ordered set of values.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    FIRST_VALUE(TotalAmount) OVER (ORDER BY OrderDate) AS FirstAmount,
    LAST_VALUE(TotalAmount) OVER (ORDER BY OrderDate) AS LastAmount
FROM Orders;
```

NTILE(n)

Distributes rows into **n** specified number of ranked groups (buckets). Rows are distributed as evenly as possible.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    NTILE(4) OVER (ORDER BY OrderDate) AS Quartile
FROM Orders;
```

CUME_DIST()

Calculates the cumulative distribution of a value in a group of values.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    CUME_DIST() OVER (ORDER BY OrderDate) AS CumulativeDistribution
FROM Orders;
```

PERCENT_RANK()

Calculates the relative rank of a row within a group of rows.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    PERCENT_RANK() OVER (ORDER BY OrderDate) AS PercentRank
FROM Orders;
```

NTH_VALUE(col, n)

Returns the value of the **n**-th row in the window frame.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    NTH_VALUE(TotalAmount, 2) OVER (ORDER BY OrderDate) AS SecondAmount
FROM Orders;
```

PERCENTILE_CONT(n) and PERCENTILE_DISC(n)

Statistical functions that calculate a percentile based on a continuous or discrete distribution. These require the **WITHIN GROUP (ORDER BY col [ASC|DESC])** clause.

```
SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY TotalAmount) OVER (ORDER BY
OrderDate) AS MedianAmount
FROM Orders;
```

```
SELECT
    category,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY price) OVER(PARTITION BY category)
AS MedianPrice
FROM #products;
```

Loops

Procedural loops are supported natively via C# execution within the ETL-SQL evaluator to handle repetition.

BEGIN ... END

A Block wrapper that allows for multiple statements inside flow controls.

IF ... ELSE IF ... ELSE

Branches data operations conditionally.

```
IF @amount > 1000
BEGIN
    INSERT INTO #high_value SELECT * FROM #sales WHERE amount > 1000;
END
ELSE IF @amount > 500
```



```
BEGIN
    INSERT INTO #mid_value SELECT * FROM #sales WHERE amount > 500;
END
ELSE
BEGIN
    INSERT INTO #low_value SELECT * FROM #sales;
END;
```

WHILE

Repeats a statement or block while a specified condition is true.

```
DECLARE @accumulator INT = 0;
WHILE @accumulator < 5
BEGIN
    SET @accumulator = @accumulator + 1;
END;
```

FOR

Iterates a variable through a numeric range with an optional **STEP**.

```
FOR @idx = 100 TO 95 STEP -1
BEGIN
    INSERT INTO #results (loop_val) VALUES (@idx);
END;
```

FOREACH

Iterates comprehensively through a designated **LIST** variable.

```
DECLARE @result_list LIST = [10, 20, 30];
FOREACH @val IN @result_list
BEGIN
    INSERT INTO #results (loop_val) VALUES (@val);
END;
```

Modular ETL (Procedures & Functions)

CREATE PROCEDURE

Defines a reusable block of ETL-SQL statements.

```
CREATE PROCEDURE ArchiveSales @olderThan DATE
AS
BEGIN
    INSERT INTO archive_db.sales SELECT * FROM sales_db.sales WHERE created_at <
@olderThan;
    DELETE FROM sales_db.sales WHERE created_at < @olderThan;
END;

EXEC ArchiveSales '2025-01-01';
```

CREATE FUNCTION

Defines a User-Defined Function (UDF) that returns a scalar value.

```
CREATE FUNCTION CalculateTax(@amount DECIMAL) RETURNS DECIMAL
AS
BEGIN
    RETURN @amount * 0.15;
END;

SELECT id, CalculateTax(price) AS tax FROM #sales;
```

DROP FUNCTION / PROCEDURE

Removes a user-defined function or procedure.

```
DROP FUNCTION [IF EXISTS] MyFunc;
DROP PROCEDURE [IF EXISTS] MyProc;
```

Transactions & Error Handling

Transactions

Supports atomic operations via a transaction stack.

- **BEGIN TRANSACTION** (or **BEGIN TRAN**)
- **COMMIT** (or **COMMIT TRAN**)
- **ROLLBACK** (or **ROLLBACK TRAN**)
- **@@TRANCOUNT**: Built-in variable returning the current nesting level.

Error Handling

- **TRY...CATCH**: Standard block for capturing runtime exceptions.
- **THROW [number, 'message', state]**: Raises a custom error.

```

BEGIN TRY
    BEGIN TRANSACTION;
    -- Operations...
    IF @errorCondition = 1 THROW 50000, 'Custom Error', 1;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT('Error: ' + ERROR_MESSAGE());
END CATCH;

```

Automation

File & Directory Operations

Specialized statements for filesystem management. Supports both a function-like syntax and a SQL-like syntax. Paths support **Connection-based Path Resolution** (e.g., `MyDir + '/file.csv'`).

Standard File Operations

- `COPY FILE <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `MOVE FILE <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `RENAME FILE <source> TO <new_name> [WITH(OVERWRITE=ON|OFF)];`
- `DELETE FILE <path>;`
- `COMPRESS FILE <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `ENCRYPT FILE <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `DECRYPT FILE <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`

Standard Directory Operations

- `CREATE DIRECTORY <path> [WITH(OVERWRITE=ON|OFF)];`
- `COPY DIRECTORY <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `MOVE DIRECTORY <source> TO <destination> [WITH(OVERWRITE=ON|OFF)];`
- `RENAME DIRECTORY <source> TO <new_name> [WITH(OVERWRITE=ON|OFF)];`
- `DELETE DIRECTORY <path>;`
- `DELETE DIRECTORY_CONTENTS <path> [WITH(RECURSIVE=ON|OFF)];`
- `COMPRESS DIRECTORY <source> TO <destination.zip> [WITH(OVERWRITE=ON|OFF)];`
- `ENCRYPT DIRECTORY <source> TO <destination> PASSWORD('<pwd>') [WITH(OVERWRITE=ON|OFF)];`
- `DECRYPT DIRECTORY <source> TO <destination> PASSWORD('<pwd>') [WITH(OVERWRITE=ON|OFF)];`

Underscore-based "Function" Syntax (Backward Compatible)

The following also support an optional 3rd parameter for `OVERWRITE` (default `ON`):

- `COPY_FILE('src', 'dest', [ON|OFF])`
- `MOVE_FILE('src', 'dest', [ON|OFF])`

- `RENAME_FILE('src', 'new_name', [ON|OFF])`
- `DELETE_FILE('path')`
- `COMPRESS_FILE('src', 'dest', [ON|OFF])`
- `ENCRYPT_FILE('src', 'dest', [ON|OFF])`
- `DECRYPT_FILE('src', 'dest', [ON|OFF])`
- `CREATE_DIRECTORY('path', [ON|OFF])`
- `COPY_DIRECTORY('src', 'dest', [ON|OFF])`
- `MOVE_DIRECTORY('src', 'dest', [ON|OFF])`
- `RENAME_DIRECTORY('src', 'new_name', [ON|OFF])`
- `DELETE_DIRECTORY('path')`
- `DELETE_DIRECTORY_CONTENTS('path', [RECURSIVE=ON|OFF])`
- `COMPRESS_DIRECTORY('src', 'dest', [ON|OFF])`
- `ENCRYPT_DIRECTORY('src', 'dest', 'pwd', [ON|OFF])`
- `DECRYPT_DIRECTORY('src', 'dest', 'pwd', [ON|OFF])`

SSH Key Pair Generation

Generate cryptographic SSH key pairs for secure file encryption and SFTP authentication.

Syntax: `CREATE SSH_KEY_PAIR('<directory_path>' [, <bits>, '<algorithm>', '<passphrase>', '<comment>']];`

- **directory_path:** The folder where the keys will be saved. (Required)
- **bits:** Key size (e.g., 2048, 3072, 4096 for RSA; 256, 384, 521 for ECDSA).
- **algorithm:** `RSA` (Default), `ECDSA`, `ED25519`.
- **passphrase:** Optional passphrase to encrypt the private key.
- **comment:** Optional comment to include in the public key.

Example:

```
-- Generate a standard RSA key pair
CREATE SSH_KEY_PAIR('C:\Keys\prod_rsa', 3072, 'RSA');

-- Generate an encrypted ECDSA key pair
CREATE SSH_KEY_PAIR('C:\Keys\prod_ecdsa', 384, 'ECDSA', 's3cr3t_pass');
```

Introspection

[!NOTE] `SHOW CONNECTIONS`, `SHOW TABLES`, and `SHOW COLUMNS` are supported in the VS Code extension and console editor UI. They are not currently available as standalone SQL statements in batch scripts.

SHOW CONNECTIONS

Displays all configured connections in the current session.

`SHOW TABLES [connection_name]`

Displays all tables in a connection.

SHOW COLUMNS <connection_name>.<table_name>

Displays all columns for a table via a specific connection.

Utility Commands

EXPLAIN

Displays the execution plan for a query — showing the join strategies, index usage, and data flow before the query runs.

```
EXPLAIN
SELECT o.OrderId, c.Name
FROM orders_db.Orders AS o
JOIN customers_db.Customers AS c ON o.CustomerId = c.Id
WHERE o.Status = 'Open';
```

HELP CONNECTION <type>

Displays connection-specific options and supported operations for the given connector type.

```
HELP CONNECTION SFTP;
HELP CONNECTION MSSQL;
HELP CONNECTION FLATFILE;
```

LINT

Statically analyzes a script for errors and best practices without executing it. Returns a result set of findings.

Syntax: **LINT** ['<script_path>'];

- With a path: analyzes the specified **.etlsql** file.
- Without a path: analyzes the current script (useful in interactive mode).

```
-- Analyze a script file
LINT 'scripts/nightly_load.etlsql';
```

PRINT

Outputs a message to the console or messages panel.

Syntax: **PRINT**(message[, show_timestamp[, format]]);

```
PRINT('Starting job...');
PRINT('Load complete', TRUE);           -- with timestamp
PRINT(GETDATE(), TRUE, 'yyyy-MM-dd'); -- formatted date
```

Job Scheduling

See [CREATE JOB](#) in Job & Profile Management for the full scheduling syntax.

```
SHOW JOB HISTORY;           -- view all job runs
SHOW JOB HISTORY NightlyArchive; -- view a specific job's history
```

LINEAGE

The lineage system provides end-to-end traceability of data movement, capturing every transformation, source-to-target mapping, and metadata tag inheritance.

Syntax:

- `LINEAGE(<target_table> [, <column_name>]) [TO '<file_path>'];`
- `SELECT * FROM LINEAGE(<target_table> [, <column_name>]);`

Variants:

1. **Console View:** `LINEAGE(#FinalAudit);` displays a hierarchical tree of sources in the console output.
2. **Column Trace:** `LINEAGE(#FinalAudit, 'UserId');` filters the trace to a single column's ancestry.
3. **Markdown Export:** `LINEAGE(#FinalAudit) TO 'reports/lineage.md';` generates a comprehensive Markdown report including a **Mermaid.js** relationship diagram and a detailed audit table.
4. **Queryable Source:** `SELECT Operation, SourceTables FROM LINEAGE(#FinalAudit);` allows you to treat the lineage audit log as a standard table for automated validation or reporting.

Queryable Columns:

- `Timestamp, Operation, TargetTable, TargetColumn, SourceTables, SourceColumns, Description, Metadata (JSON), DerivedFromDescriptions, SourceFile, Line, Column.`

Metadata Inheritance & Amalgamation

When columns are combined or transformed (e.g., `UnitPrice * Qty AS Total`), the system automatically propagates metadata:

1. **Last-Seen Wins:** The primary description (@d) and individual custom tags are inherited from the last source column in the expression that has them defined.
2. **Amalgamation:** The `DerivedFromDescriptions` field is populated with a structured list of all involved source descriptions (e.g., `UnitPrice: Base price per unit, Qty: Number of items sold`), ensuring no context is lost during transformations.

3. **Global Persistence:** All tags assigned anywhere in the lineage chain are preserved and queryable at the final destination.

File System Functions

- **FILE_EXISTS(path):** Returns **TRUE** if the specified file exists.
- **DIRECTORY_EXISTS(path):** Returns **TRUE** if the specified directory exists.
- **FILE_LIST(path[, recursive]) / DIRECTORY:** (Table-valued) Returns a table of files in a local directory (Columns: **Name**, **Path**, **Extension**, **Size**, **LastModified**).
- **REMOTE_FILE_LIST(conn_name[, path]):** (Table-valued) Returns a list of files from a remote connection (SFTP/FTP/Blob).

Lineage & Metadata Functions

- **GET_TAGS(target_table[, column_name]):** Returns a **LIST** of all custom metadata tag names defined for a table or specific column.
- **GET_TAG_VALUE(table_name, column_name, tag_name):** Returns the string value of a specific metadata tag.

Example:

```
DECLARE @tags LIST = GET_TAGS('#TaggedUsers', 'UserId');
IF 'sensitive' IN @tags
BEGIN
    PRINT('Warning: Table contains sensitive data.');
```

```
END;
```

Attach arbitrary metadata tags to columns and tables using special comment blocks. Tags are captured by the lineage system for auditing, governance, and UI documentation. **@d:** is the reserved description tag.

Column Tags — placed immediately after a column expression in a **SELECT** list:

```
col_name /* @d: Description text; @tag: value; @another: value2; */
```

Table Tags — placed immediately after a table reference in **FROM** or **JOIN**:

```
FROM table_name /* @owner: TeamA; @sensitivity: high; */
```

Example:

```
SELECT
    UserId /* @d: Internal user ID; @sensitive: true; */,
    UserName /* @d: Full name of the user; @owner: Chuck; */
INTO #TaggedUsers
FROM m.Users /* @owner: SecurityTeam; */;
```

Interactive Terminal Editor (ui edit)

Experience a modern, terminal-based development environment designed for high-productivity ETL scripting. The editor provides real-time feedback, intelligent assistance, and professional-grade text editing features directly in your console.

Launching the Editor

Open any `.etlsql` script (or start a new one) by passing the `--ui edit` flag to the application:

```
dotnet run --project src/ETL-SQL.App -- --ui edit MyScript.etlsql
```

Key Features

- **Live Results Grid:** Interactive paging and multi-result set navigation. Focus the results pane with `F3` to scroll through large datasets.
- **Vibrant Syntax Highlighting:** Context-aware coloring for DML, DDL, Control Flow, and specific ETL keywords.
- **Intelligent Autocomplete:** Deep integration with data source schemas, variables, and file systems. Trigger manually with `Ctrl+Space`.
- **Execution Profiling:** Toggle the performance panel with `F4` to see millisecond-level metrics for every step of your script.
- **Multi-Cursor Editing:** Edit multiple lines simultaneously using `Alt+Up/Down` to add vertical cursors.

Keyboard Shortcuts

Shortcut	Action
F1	Help Overlay - View all shortcuts and commands.
F5	Run Script - Execute the entire current buffer.
Shift+F5	Run Statement - Execute only the line at the cursor.
Ctrl+Space	Autocomplete - Trigger IntelliSense for keywords, tables, or columns.
Ctrl+S	Save - Persist changes to disk (use <code>Shift+Ctrl+S</code> for Save As).
Ctrl+I / Alt+F	Format Document - Standardize indentation and keyword casing.
F3	Focus Toggle - Switch focus between the Editor and the Results Grid.
F4	Performance Toggle - Switch between Results and Performance metrics.
Ctrl+F	Find - Search for text within the current buffer.
Ctrl+H	Replace - Search and replace text (supports case-insensitive matching).
Ctrl+P	Export Results - Save the last result set to a CSV file.

Shortcut	Action
Ctrl+Z / Ctrl+Y	Undo / Redo - Revert or re-apply text changes.
Ctrl+D	Duplicate Line - Copy the current line immediately below.
Ctrl+K	Delete Line - Remove the current line entirely.
Alt+Up/Down	Multi-Cursor - Add another cursor on the line above or below.
Esc	Clear - Remove multi-cursors or hide overlays.
Ctrl+Q	Exit - Close the editor (prompts to save if modified).

VS Code Extension Support

For developers who prefer a full IDE experience, ETL-SQL includes a dedicated Visual Studio Code extension powered by a high-performance Language Server.

Installation

The extension is currently available in **Developer Mode**. To install it locally:

1. Open PowerShell as **Administrator**.
2. Create a directory junction from the source folder to your VS Code extensions directory:

```
New-Item -ItemType Junction -Path "$HOME\.vscode\extensions\etl-sql-vscode"
-Target "C:\Users\chuck\scratch\ETL-SQL\src\etl-sql-vscode"
```

3. Restart Visual Studio Code.

Configuration

Go to VS Code Settings (**Ctrl+,**) and search for **ETL-SQL** to configure the following:

- **Server Path:** The absolute path to the **ETL-SQL.LanguageServer.exe** binary.
- **Executable Path:** The absolute path to the main **ETL-SQL.exe** engine.

Features

- **Real-time Diagnostics:** Syntax and linter errors appear in the "Problems" tab as you type.
- **Lineage Hover:** Hover over any column or table to see a visual graph of its ancestry and metadata.
- **Smart Formatting:** Use **Alt+Shift+F** to format your scripts according to engine standards.
- **Direct Execution:** Run the current script or selection using integrated command palette actions.

Native SQL Pushdown & Performance

ETL-SQL is designed with a "**Pushdown-First**" philosophy. Whenever possible, the engine delegates computation to the source database system to minimize data movement and leverage remote indexing.

How Automatic Pushdown Works

The engine analyzes your script and automatically identifies blocks that can be executed as a single unit on the remote server.

- **SELECT & Filters:** Simple **SELECT** statements with **WHERE** and **JOIN** clauses on a single database connection are pushed down entirely.
- **INSERT...SELECT:** If both the source and target tables reside on the same database connection, the engine executes a **INSERT INTO ... SELECT ...** statement directly, resulting in **Zero-Copy** transfers.
- **MERGE:** Remote pushdown is supported for MSSQL when both source and target are on the same connection.
- **UPDATE & DELETE:** These are automatically pushed to the remote server using optimized SQL.

Manual Pushdown (**EXECUTE ... BEGIN ... END**)

In cases where you need to use highly specific database features (like Recursive CTEs, specialized hints, or complex stored procedures) that may not be fully represented in ETL-SQL syntax, you can use a manual pushdown block:

```
-- This block is passed EXACTLY as written to the 'prod_db' connection
EXECUTE prod_db
BEGIN
    WITH RecursiveCTE AS (
        SELECT id, parent_id FROM Categories WHERE parent_id IS NULL
        UNION ALL
        SELECT c.id, c.parent_id FROM Categories c
        INNER JOIN RecursiveCTE r ON c.parent_id = r.id
    )
    SELECT * INTO #CategoryHierarchy FROM RecursiveCTE;
END;
```

Capturing Pushdown Results

You can capture the output of a pushdown block into a local temporary table using the **INTO** syntax:

```
EXECUTE my_db INTO #RemoteStats
BEGIN
    SELECT Category, COUNT(*) as Total
    FROM Sales
    GROUP BY Category;
END;

-- #RemoteStats is now available as a standard ETL-SQL result set
SELECT * FROM #RemoteStats ORDER BY Total DESC;
```

Verification

To verify if a statement was pushed down, check the execution logs (or the "Messages" tab in the UI). You look for the line: **Strategy: Remote SQL Pushdown (Insert from Select)** or similar "Pushdown" indicators. If these are absent, the engine is streaming data through local memory.